Curs 7 - Regularizare în Regresie Liniară

ADS1

Teodor-Ștefan Bîldea





Planul Cursului

Overfitting/Supraadaptare

Selectarea Modelului

Cross Validation/Validare încrucișată

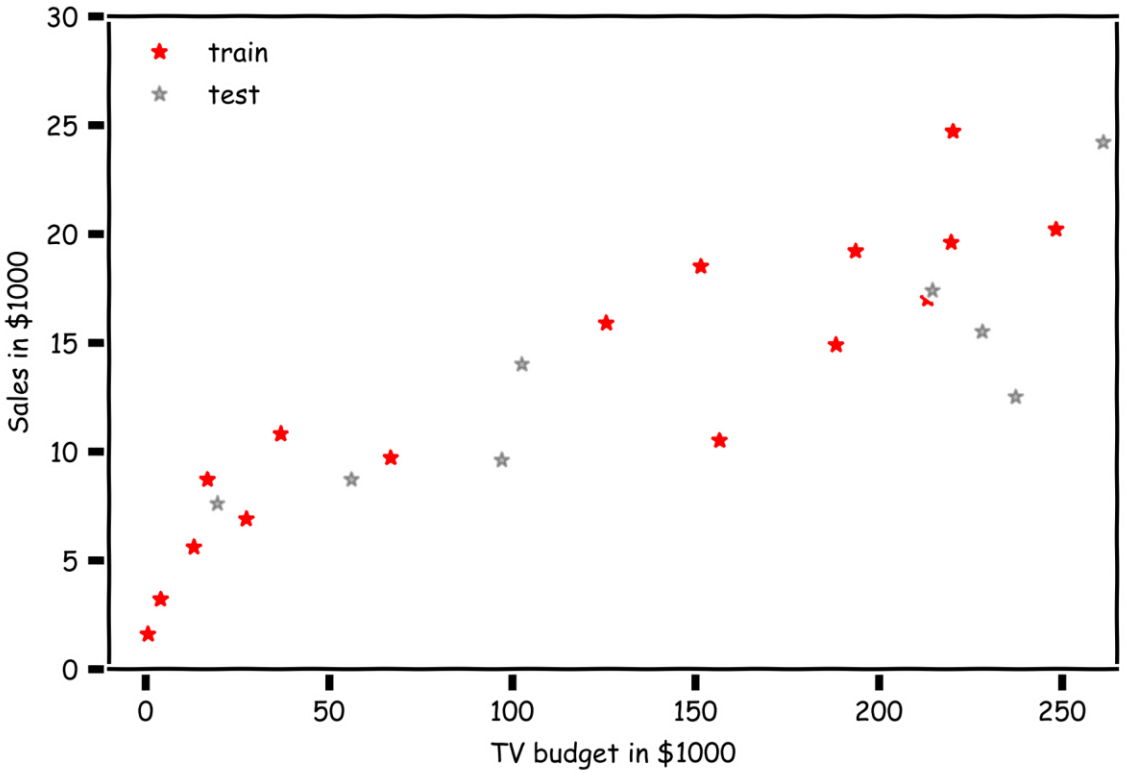
Bias vs Varianță

Regularizare: LASSO și Ridge

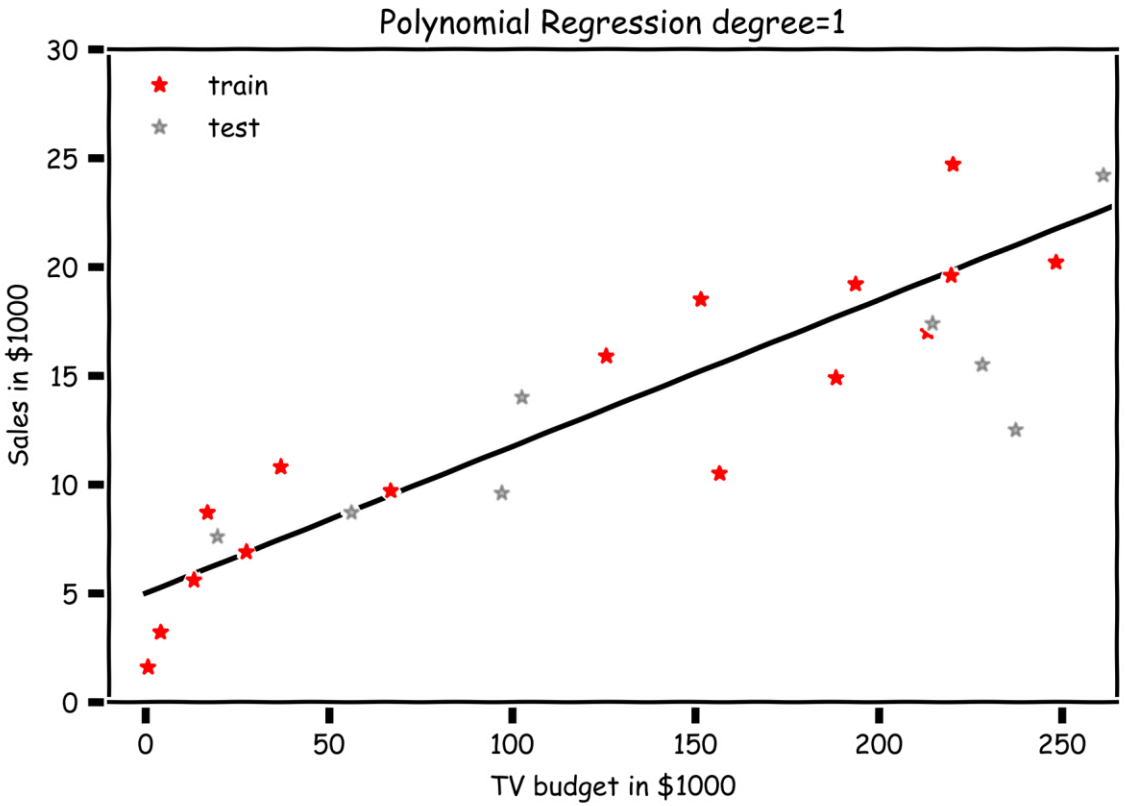
Metod de Regularizare: O Comparație

OVERFITTING/SUPRAADAPTARE

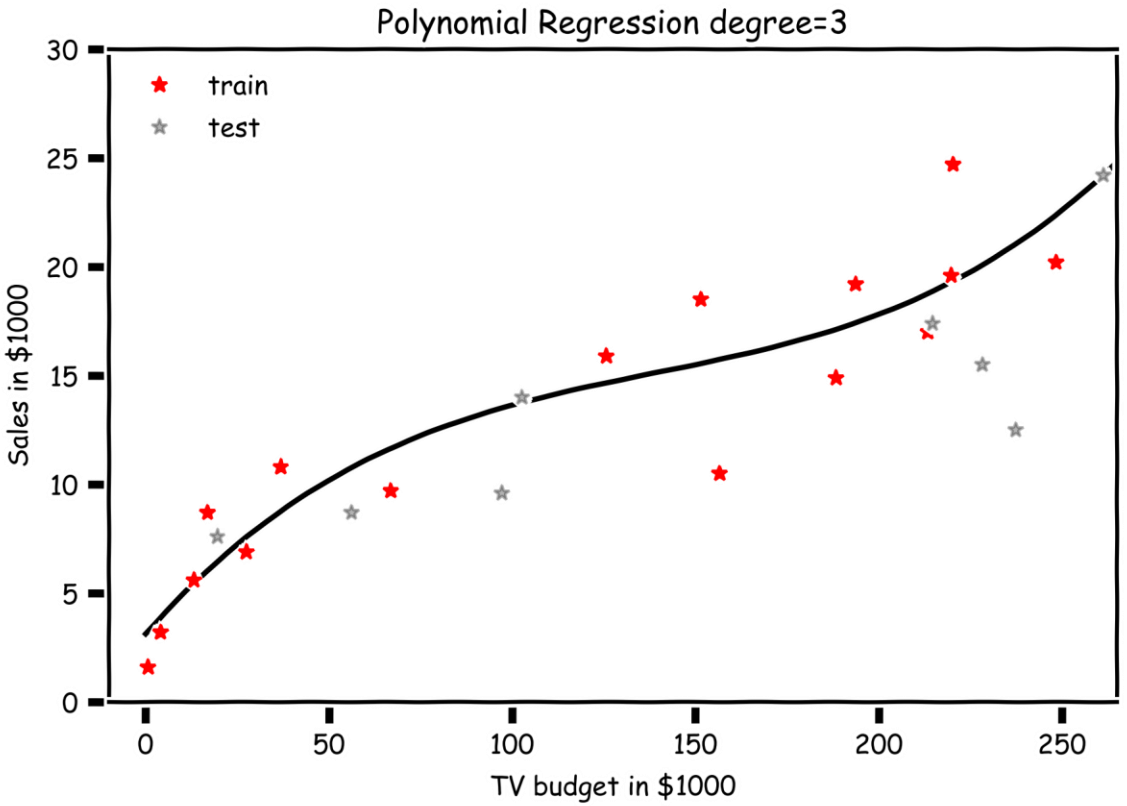
Overfitting



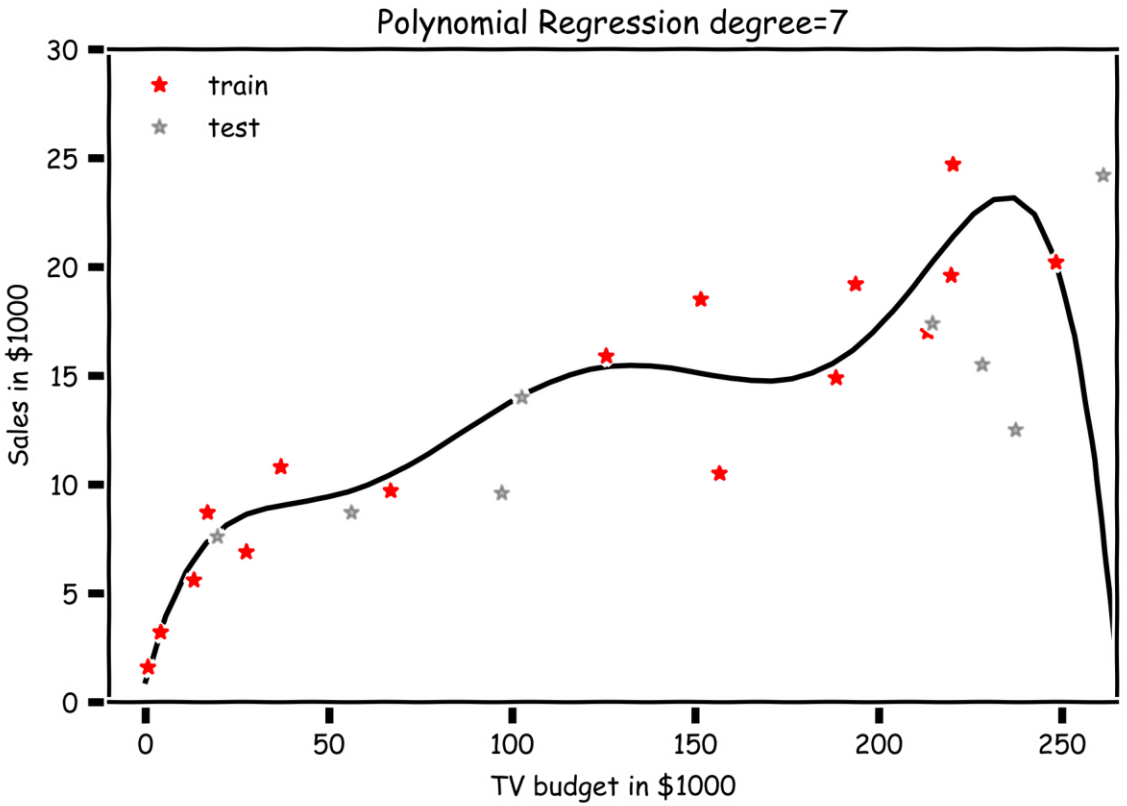
Overfitting



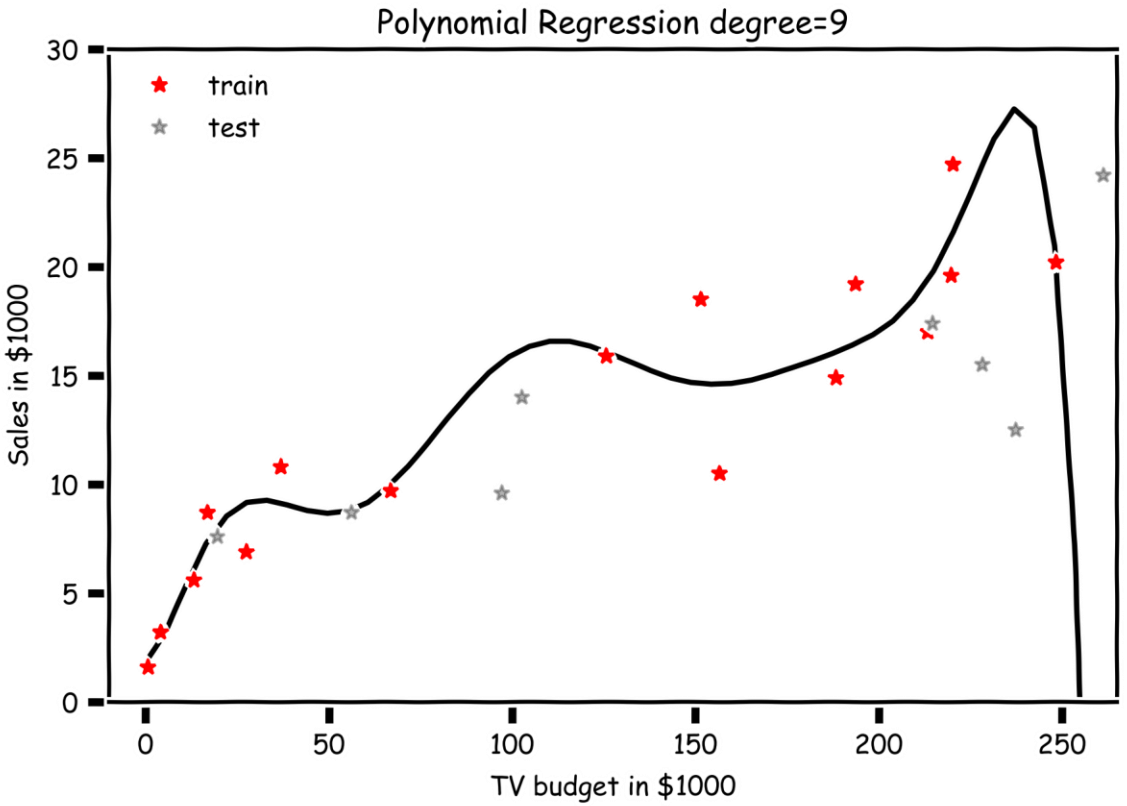
Overfitting



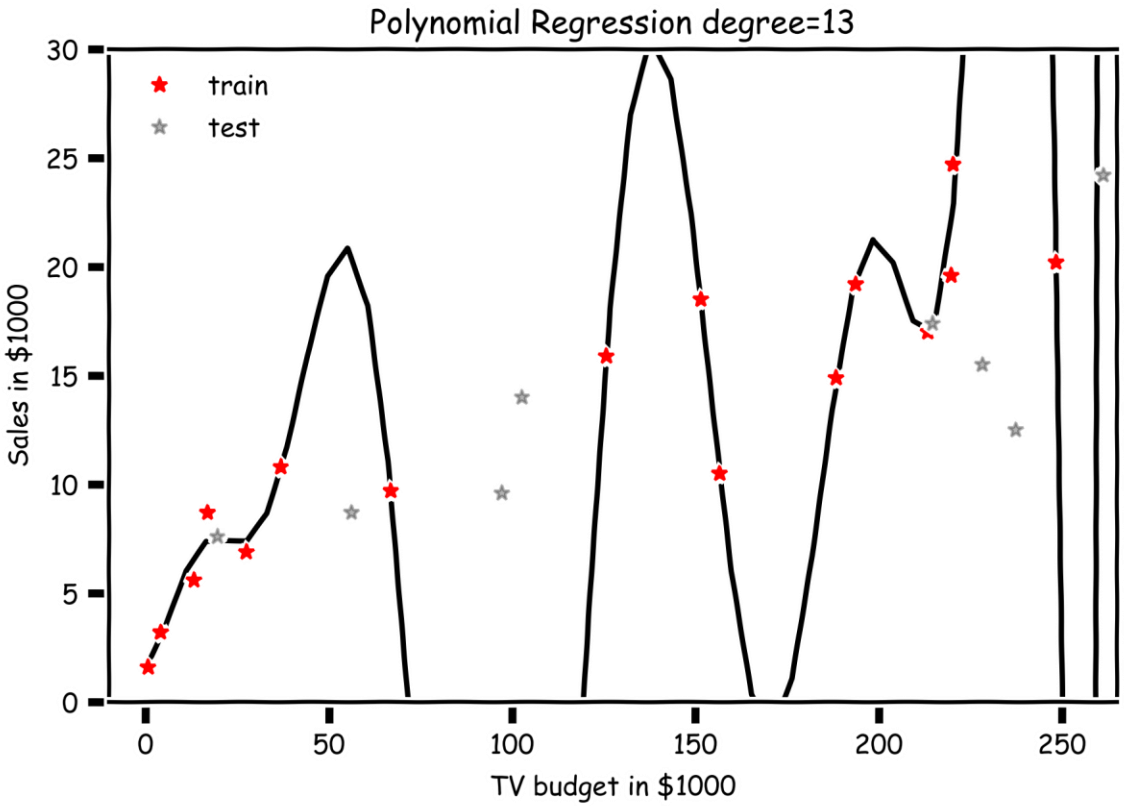
Overfitting



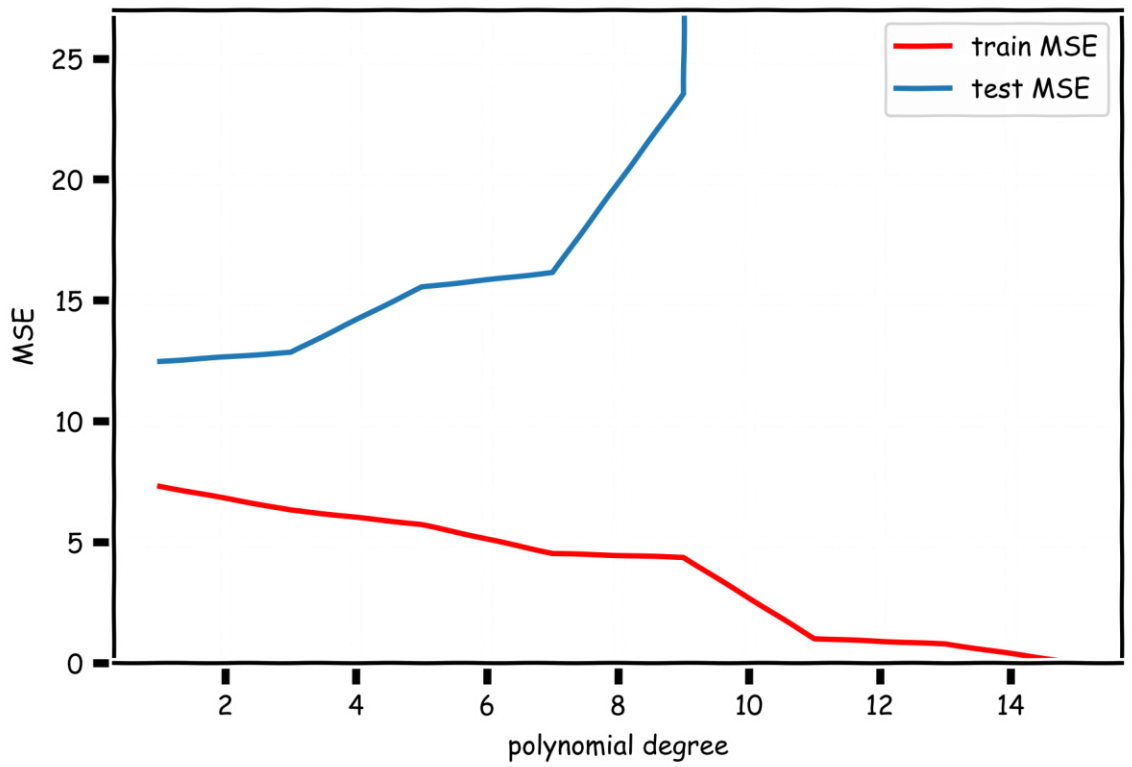
Overfitting



Overfitting



Validation


train MSE
test MSE
MSE
0
5
10
15
20
25
2
4
6
8
10
12
14
polynomial degree

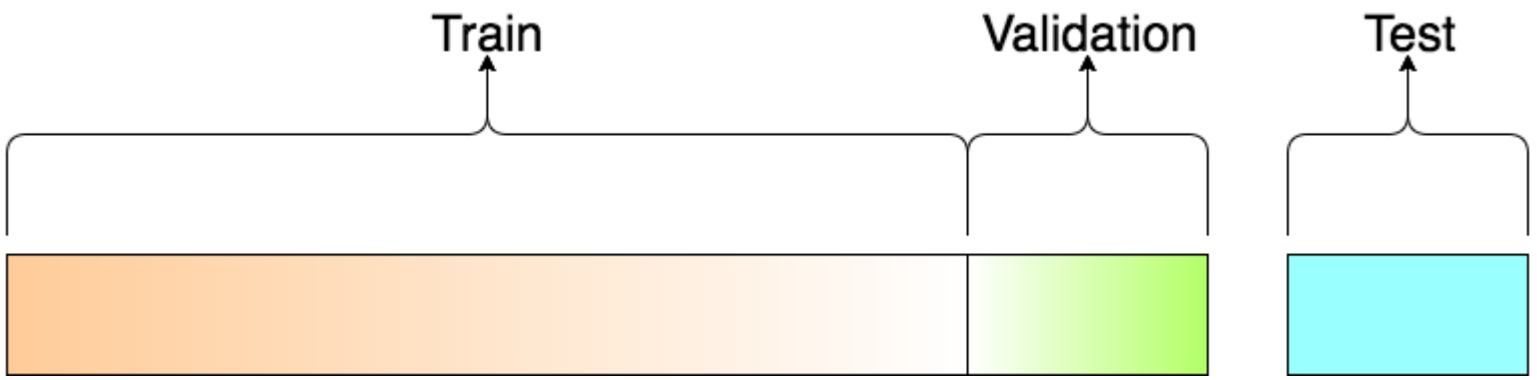
Antrenare-Validare-Testare

Întrebare:

Cum ați raporta performanța modelului?

R2_test = 0.52

R2_train(grad=1) = 0.83



SELECETAREA MODELULUI

Selectarea Modelului

Selecția modelului este aplicarea unei metode algoritmice pentru a balansa complexitatea modelului cu eroare pe datele de antrenament/test, de ex. alegerea unui subset de predictori, alegerea gradului modelului polinomial etc.

O motivație puternică pentru efectuarea selecției modelului este de a evita supraadaptarea, ceea ce am văzut că se poate întâmpla când:

- sunt prea mulți predictori:
  - spațiul caracteristicilor are o dimensionalitate ridicată (multe coloane)
  - gradul polinomial este prea mare
- **valorile** coeficienților sunt **prea extreme (nu am văzut asta încă)**

Selectarea Modelului

Întrebare:

Câte modele diferite pot exista atunci când se iau în considerare un număr de J predictori?

Selectarea Modelului

Exemplu: 3 predictori (X_1, X_2, X_3)

- Model cu 0 predictori (de ex. Media):

  M0:

- Modele cu 1 predictor:

  M1: X_1

  M2: X_2

  M3: X_3

- Modele cu 2 predictori:

  M4: $\{X_1, X_2\}$

  M5: $\{X_2, X_3\}$

  M6: $\{X_3, X_1\}$

- Modele cu 3 predictori:

  M7: $\{X_1, X_2, X_3\}$

2^J Modele

Alegere pas cu pas a predictorilor și validare îmcrucișată

Selectarea subseturilor optime de predictori (inclusiv alegerea gradului pentru modele polinomiale) prin:

- selecție pas cu pas - construirea iterativă a unui subset optim de predictori prin optimizarea unei metrici fixate de evaluare a modelului,
- validare - selectarea unui model optim prin evaluarea fiecărui model pe setul de validare.

De asemenea, vom aborda problema descurajării valorilor extreme în parametrii modelului mai târziu.

Alegere pas cu pas a predictorilor: metoda selecției directe

Forward Seletion

În selecția directă, găsim un set „optim” de predictori prin construirea iterativă a setului nostru.

1. Pornim cu mulțimea vidă P0, construim modelul nul M0.

2. Pentru $k = 1, \dots, J$:

2.1 Fie M_{k-1} modelul construit pe cea mai bună selecție de $k-1$ predictori, P_{k-1}.

2.2 Selectăm predictorul X_{n_k}, care să nu fie înP_{k-1}, așa încât $P_k = X_{n_k} \cup P_{k-1}$ optimizează o metrică fixată (poate fi una din p -value, F -stat; MSE pe setul de validare, R^2, sau AIC/BIC pe setul de testare).

2.3 Fie M_k modelul construit din submulțimea optimă P_k.

3. Alegem modelul M din $\{M_0, M_1, \dots, M_J\}$ care optimizează metrica aleasă

Alegere pas cu pas a predictorilor: Complexitate de Calcul

Câte modele vom evalua?

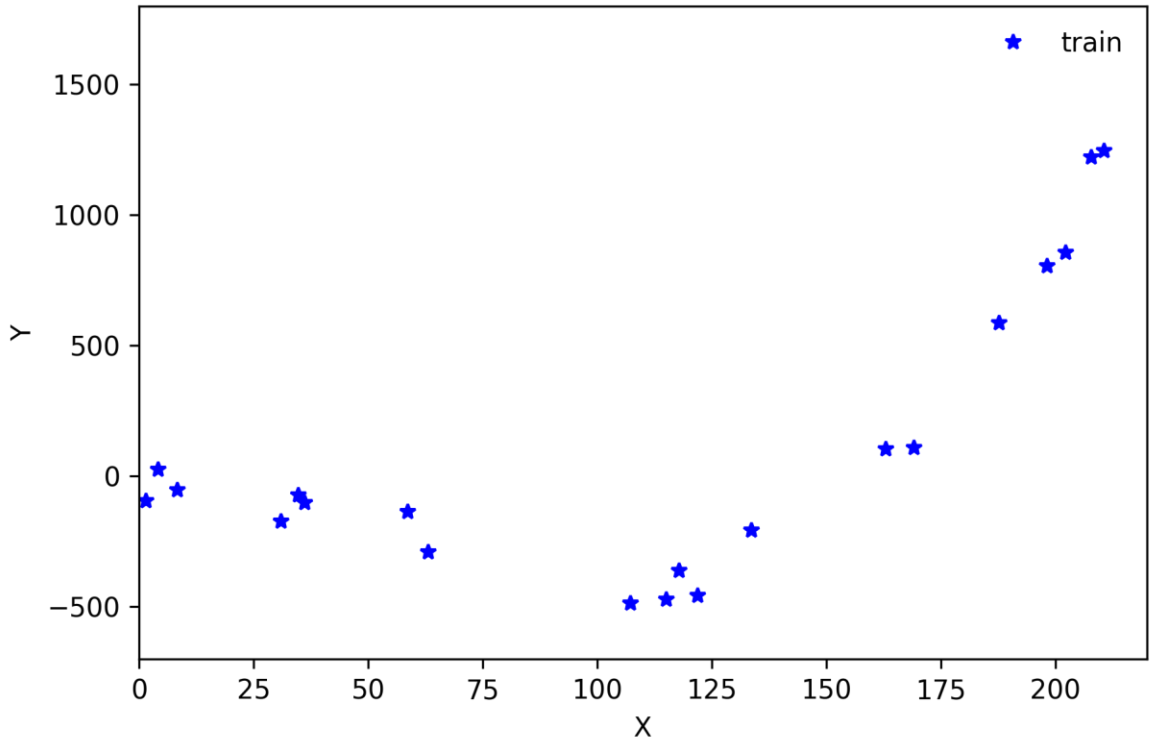
- Pas 1, ***J* Modele**
- Pas 2, *J-1* **Modele** (adăugam 1 predictor din *J-1* posibile)
- Pas 3, *J-2* **Modele**(adăugam 1 predictor din *J-2* posibile)
- …

$$O(J^2) \ll 2^J$$ pentru valori mari ale lui *J*

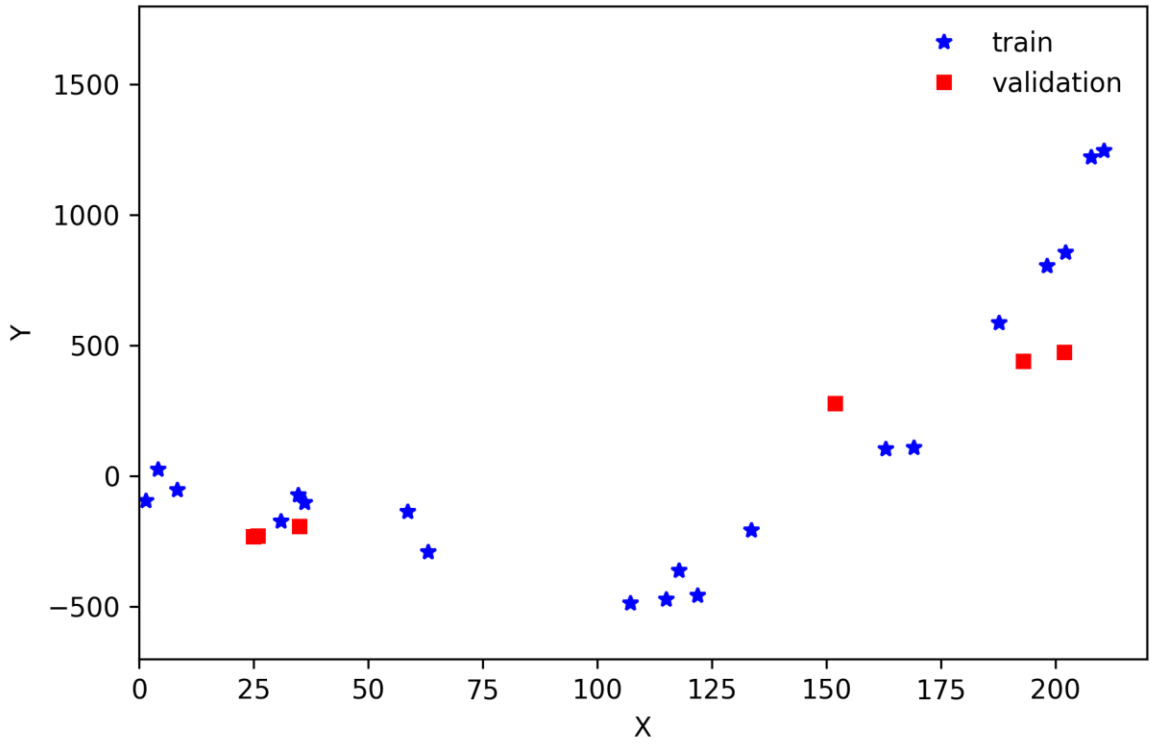
- Pentru un număr de coloane J mare (> 100) devine oricum prohibitiv acest procedeu. În mod curent se preferă modele care oferă o alternativă – vom vedea mai jos.

CROSS VALIDATION/ VALIDARE ÎNCRUCIȘATĂ

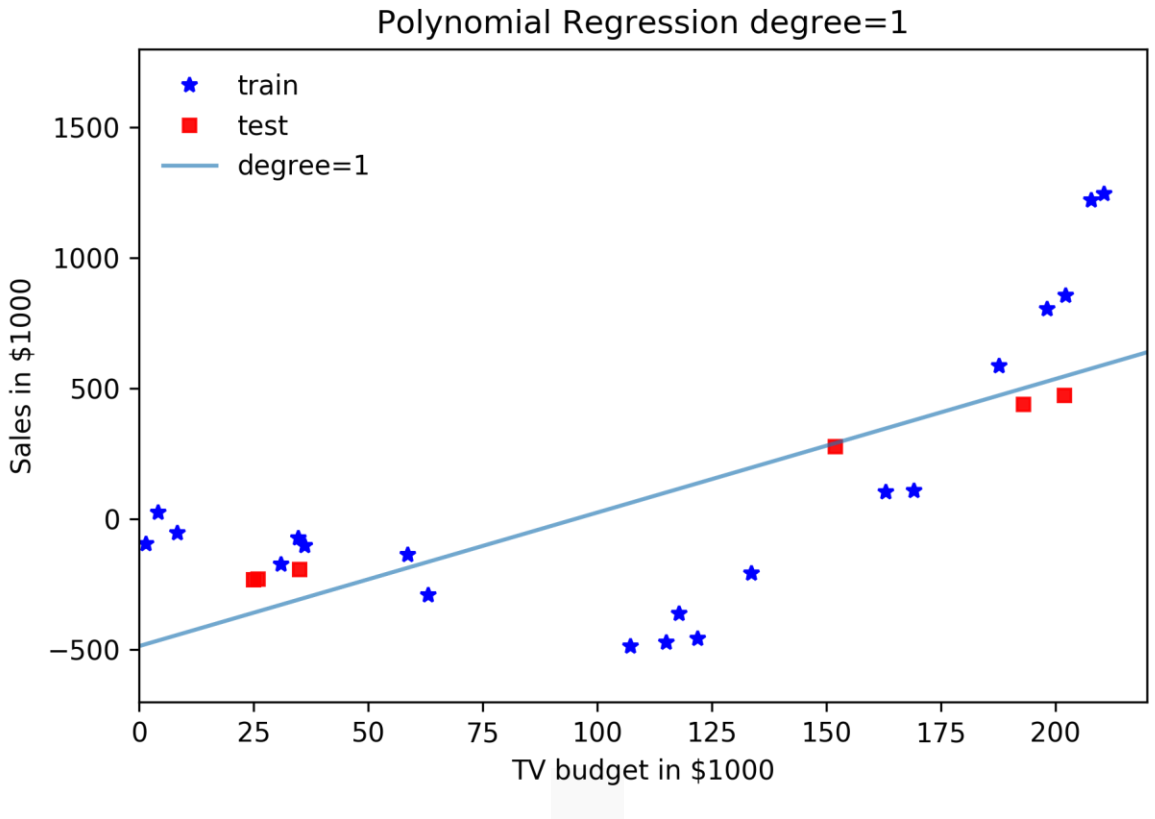
Cross Validation/Validare Încrucișată



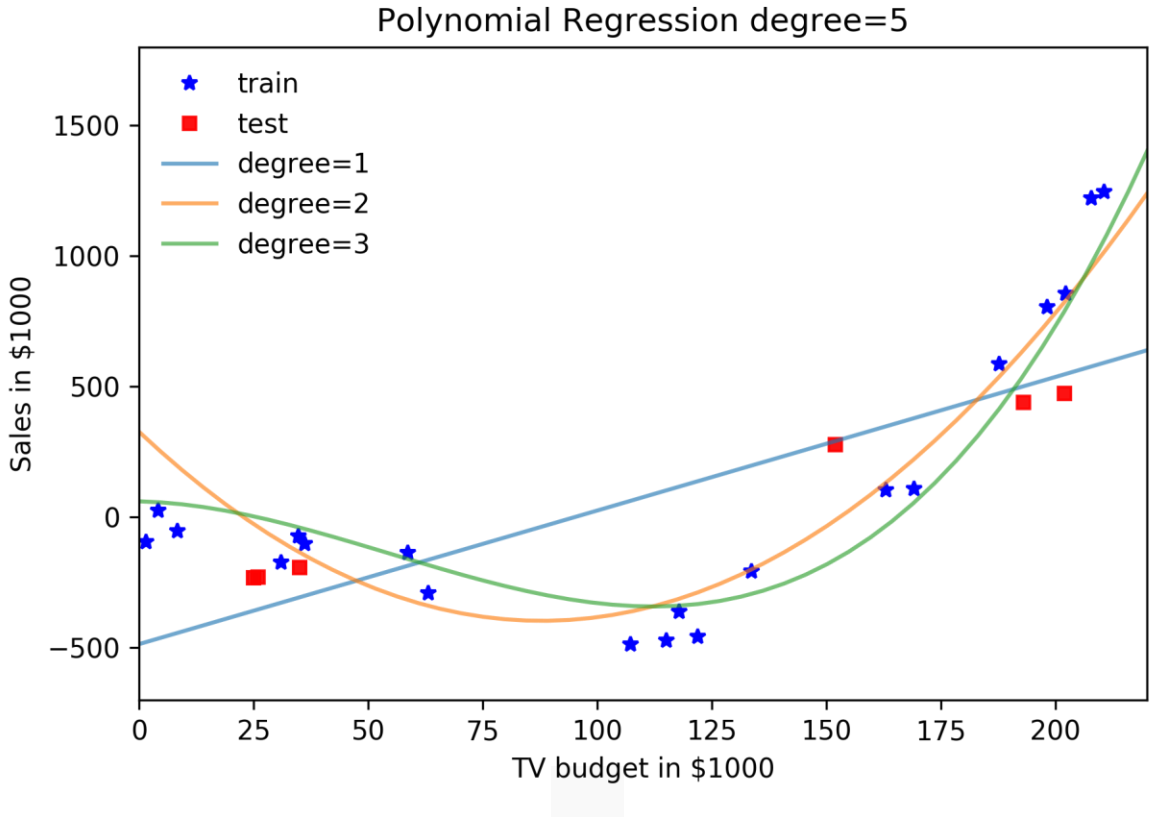
Cross Cross Validation/Validare Încrucișată



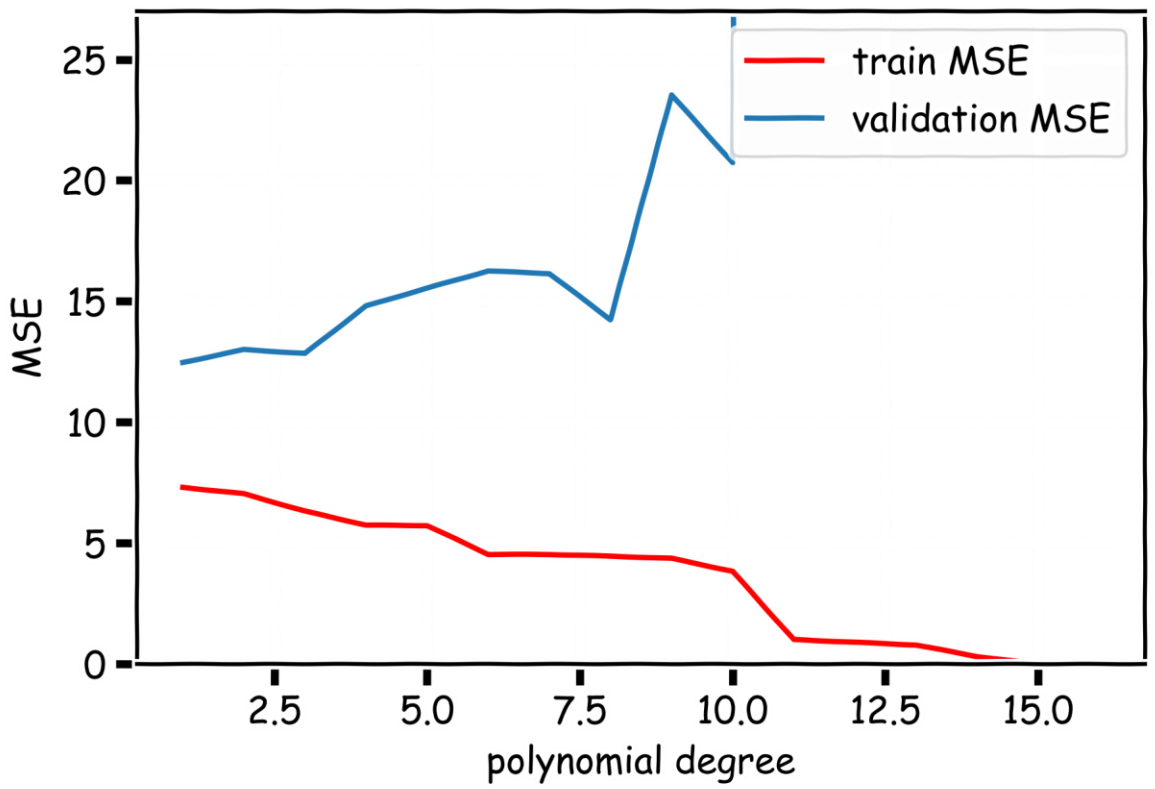
Cross Validation/Validare Încrucișată Validation



Cross ValidCross Validation/Validare Încrucișatăaation



Validare


train MSE
validation MSE
MSE
25
20
15
10
5
0
2.5
5.0
7.5
10.0
12.5
15.0
polynomial degree

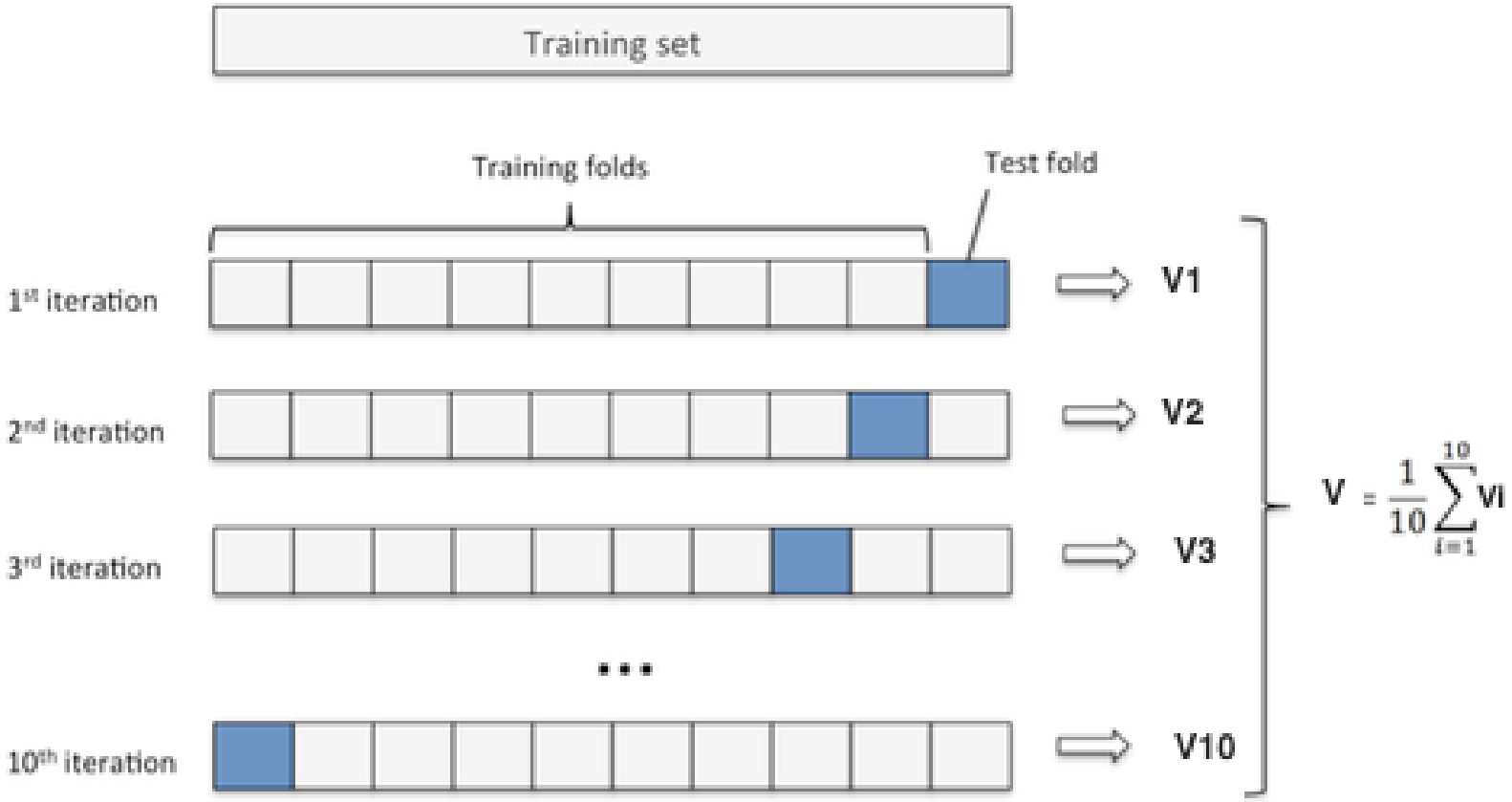
Cross Validation/Validare Încrucișată: Motivație

Utilizarea unui **singur set de validare** pentru a selecta dintre mai multe modele poate fi problematică - există posibilitatea de a se **supraadapta setului de validare** (***overfitting***).

O soluție la problemele ridicate prin utilizarea unui singur set de validare este evaluarea fiecărui model pe **mai multe seturi de validare** și media performanței validării.

Se poate împărți **aleatoriu** setul de antrenament în antrenament și validare de mai multe ori, dar crearea aleatorie a acestor seturi poate crea scenariul în care caracteristicile importante ale datelor nu apar niciodată în extragerile noastre aleatorii.

Cross Validation/Validare Încrucișată



K-Fold Cross Validation/Validare Încrucișată de K ori

Pentru a ne asigura că fiecare observație din setul de date este inclusă în cel puțin un set de antrenament și cel puțin un set de validare, utilizăm **validarea de K ori**:

- Se împart datele în *K* submulțimi de înregistrări în mod uniform, $\{C_1, \dots, C_K\}$ (toate cam aceeiași proporție de date)
- creăm numărul K de perechi set de antrenament / validare, folosind una dintre cele M bucăți pentru validare și restul pentru antrenament.

Modelul antrenat pe fiecare set de antrenament în parte, notat $\hat{f}_{C_{-i}}$, este apoi evaluat pe setul de validare corespunzător , $\hat{f}_{C_{-i}}(C_i)$. ***Validarea încrucișată*** este performanța medie a modelului pentru toate seturile de validare:

$$CV(\text{Model}) = \frac{1}{K}\sum_{i=1}^{K} L(\hat{f}_{C_{-i}}(C_i))$$

unde *L* este funcția ce va fi optimizată.

Selectarea Predictorilor: Cross Validation/Validare Încrucișată

Întrebare: Care este raportul corect dintre mărimile mulțimilor de anternament / validare / testare, cum aleg K?

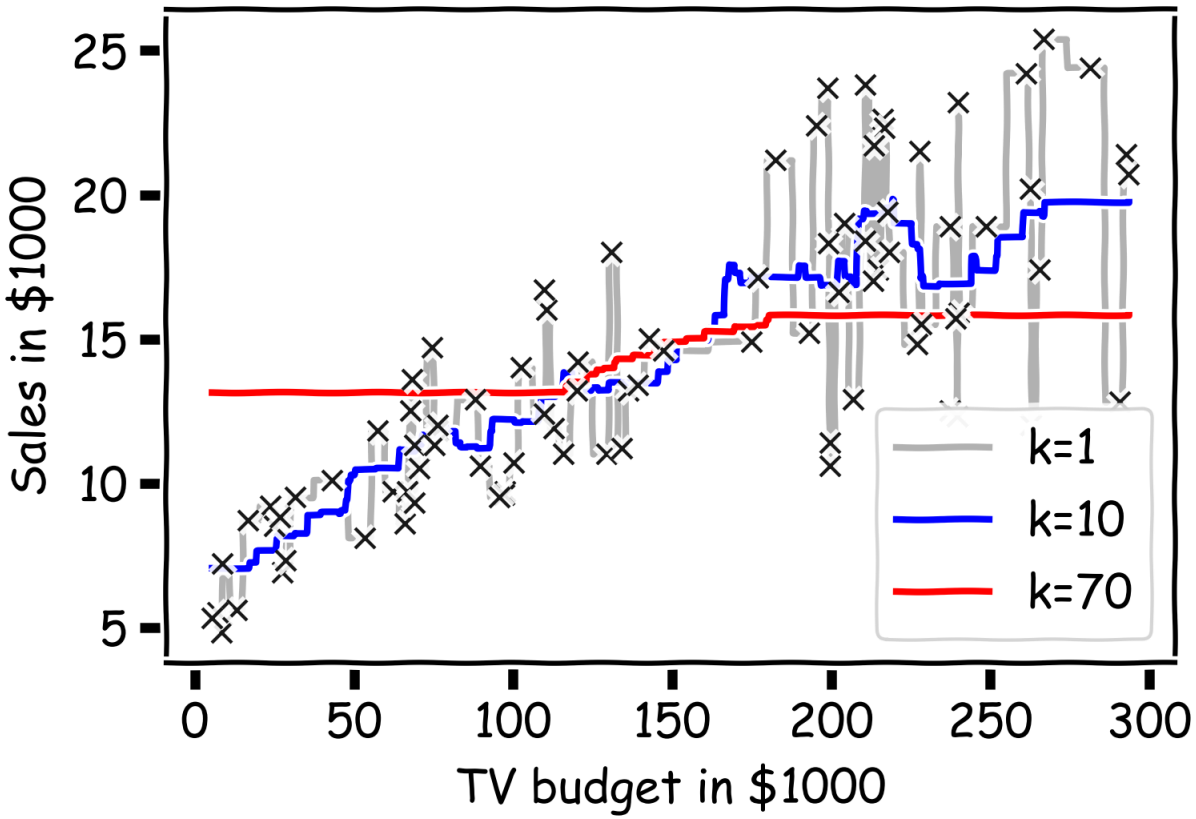
- Nu există o regulă. O bună pratică pentru date rezonabile (zeci de mii de înregistrări – 80% antrenament, 20% test sau 80% antrenament, 10% validare, 10% test)

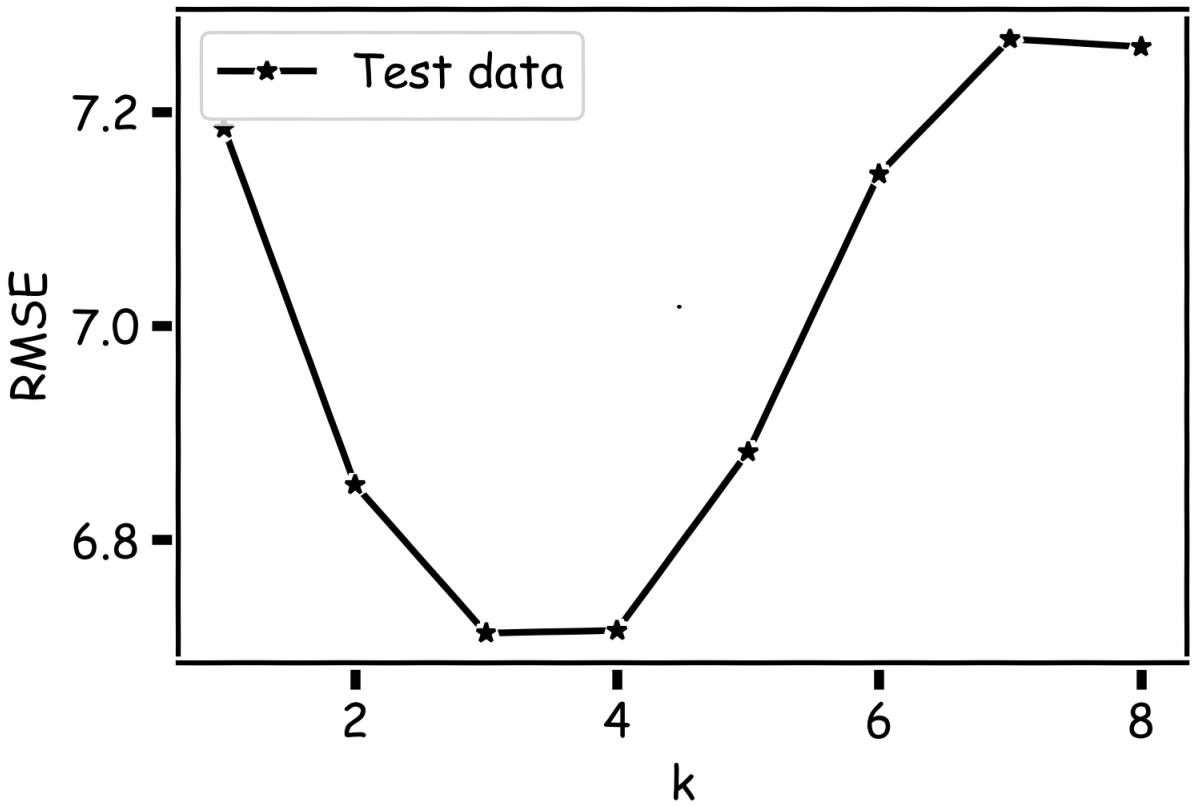
- Pentru date mari, 20% poate însemna foarte mult – la 100 mii/1 mil înregistrări de exemplu se folosește deja 98% 1% 1%.

Putem încadra problema selecției gradelor pentru modelele polinomiale ca o problemă de selecție a predictorilor:

care din predictorii $\{x, x^2, \dots, x^m\}$, are trebui selectați pentru model?

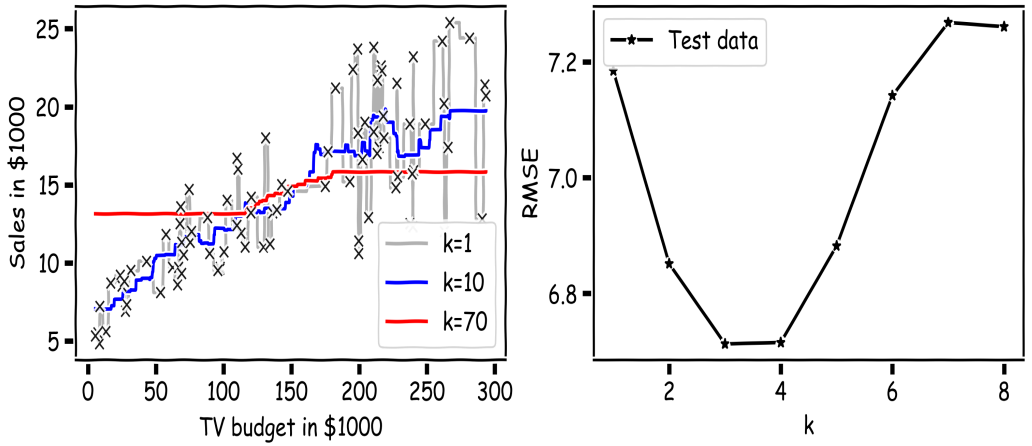
Din nou kNN


Sales in $1000
25
20
15
10
5
0
50
100
150
200
250
300
TV budget in $1000
k=1
k=10
k=70


RMSE
7.2
7.0
6.8
2
4
6
8
k
Test data

Din nou kNN

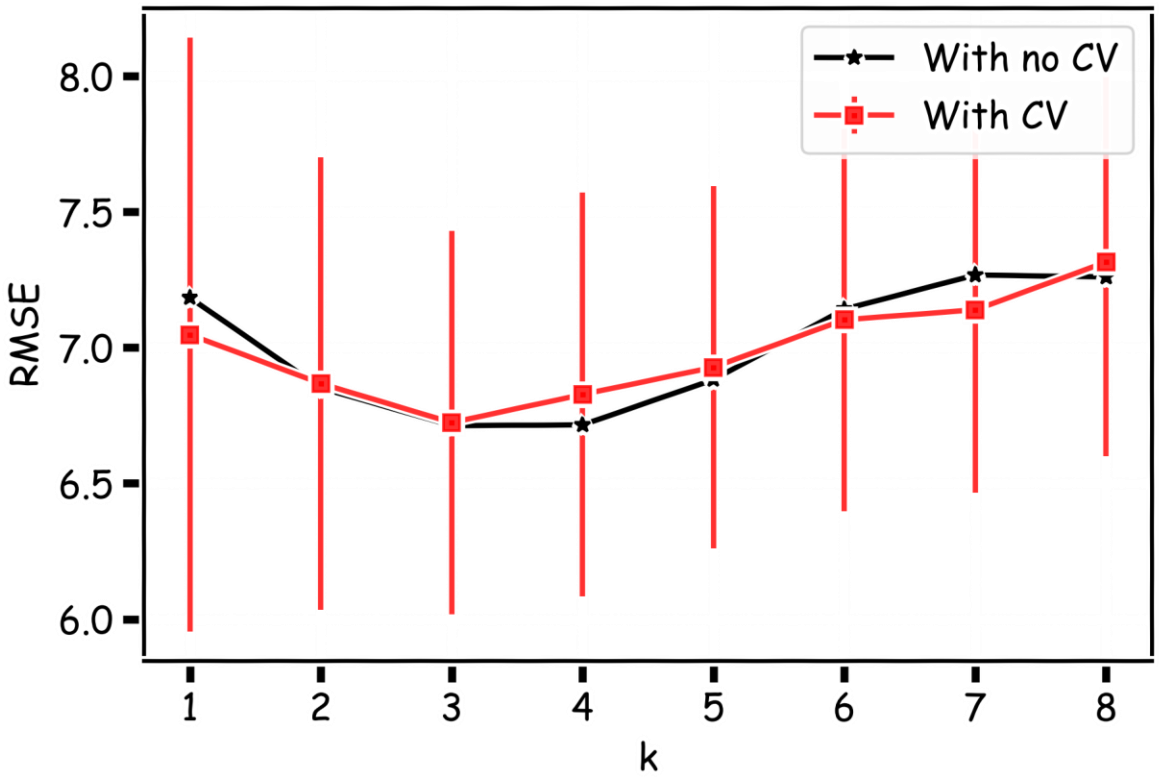
Recapitulăm primul nostru model simplu, intuitiv, non-parametric pentru regresie – modelul kNN. Am văzut că este foarte important să selectăm un k adecvat pentru date.



Când *k* este prea mic, modelul este foarte sensibil la zgomot (deoarece o nouă predicție se bazează pe foarte puțini vecini observați), iar când k este prea mare, modelul tinde să facă predicții constante.

Un mod principial de a alege *k* este prin **validare încrucișată**

Spargere CV – K = 100



BIAS VS VARIANȚĂ

Bias vs Variance (sub-ajustare vs supra-ajustare)

Bias (eroare de părtinire / sub-ajustare)

Bias mare = model prea simplu, care nu surprinde relațiile reale din date.

Cum se manifestă:
- Erori mari pe setul de antrenare.
- Model care “nu învață” suficient.
- Prezice aproape o medie constantă.
- Rezidualele au structură clară, modelul nu surprinde pattern-uri.

Exemple de modele cu bias mare:
- Regresie liniară pentru un fenomen puternic neliniar.

Variance (eroare de variație / supra-ajustare)

Variance mare = model prea complex, care învață și zgomotul din date.

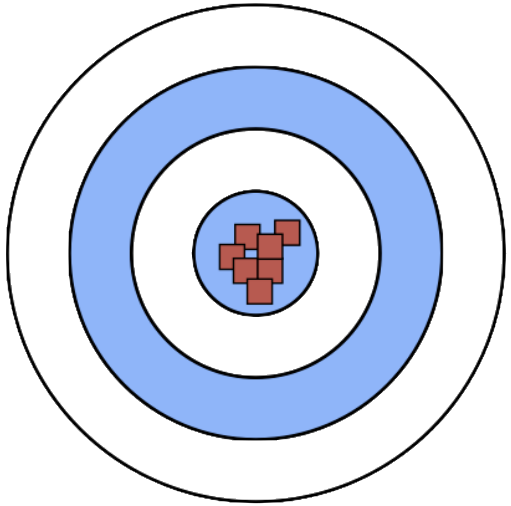
Cum se manifestă:
- Performanță foarte bună pe setul de antrenare.
- Performanță slabă pe setul de test / validare.
- Predicții instabile: mici schimbări în date → mari schimbări în predicții.
- Curba de învățare cu decalaj mare între train și test.

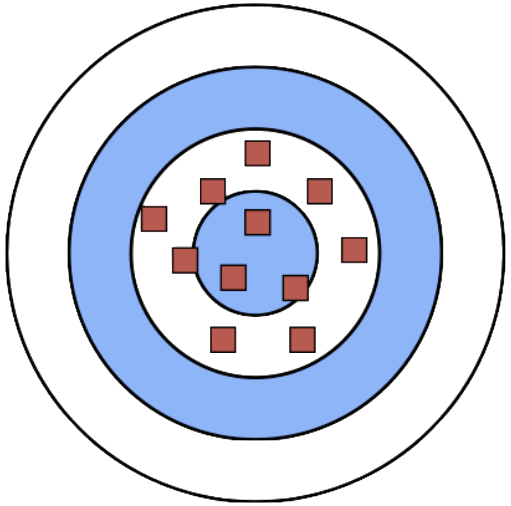
Exemple de modele cu variație mare:
- Arbori de decizie foarte adânci.

Supra-ajustare mică
(=>precizie mare)

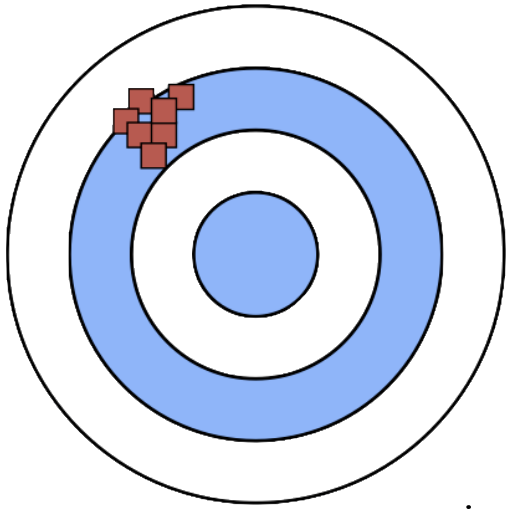
Supra-ajustare mare
(=>acuratețe mică)

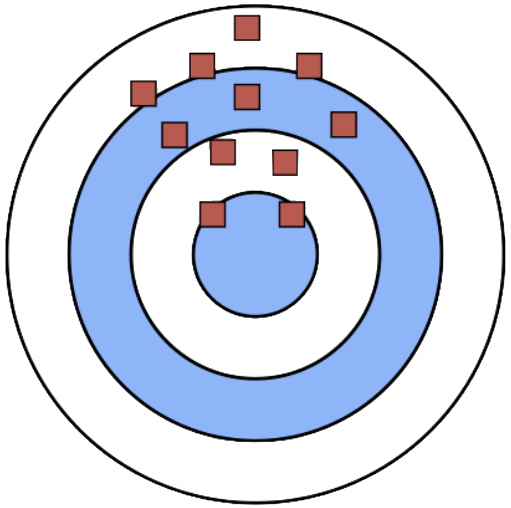
Sub-ajustare mică
(=>acuratețe mare)





Sub-ajustare mare
(=>acuratețe mică)





Tratament Bias/sub-ajustare

🛠️ **Cum reduci bias-ul (model prea simplu)?**

📌 *Scop:* modelul să poată învăța mai bine structura datelor.

Crești complexitatea modelului:

- Folosești un model mai flexibil (ex. treci de la regresie liniară la polinomială)
- Adaugi mai mulți predictori relevanți.
- Folosești modele non-lineare (XGBoost, Random Forest, SVM RBF).
- Scazi coeficientul de regularizare

Tratament Variance/supra-ajustare/supra-adaptare

🛠️ **Cum reduci variația (model prea complex)?**

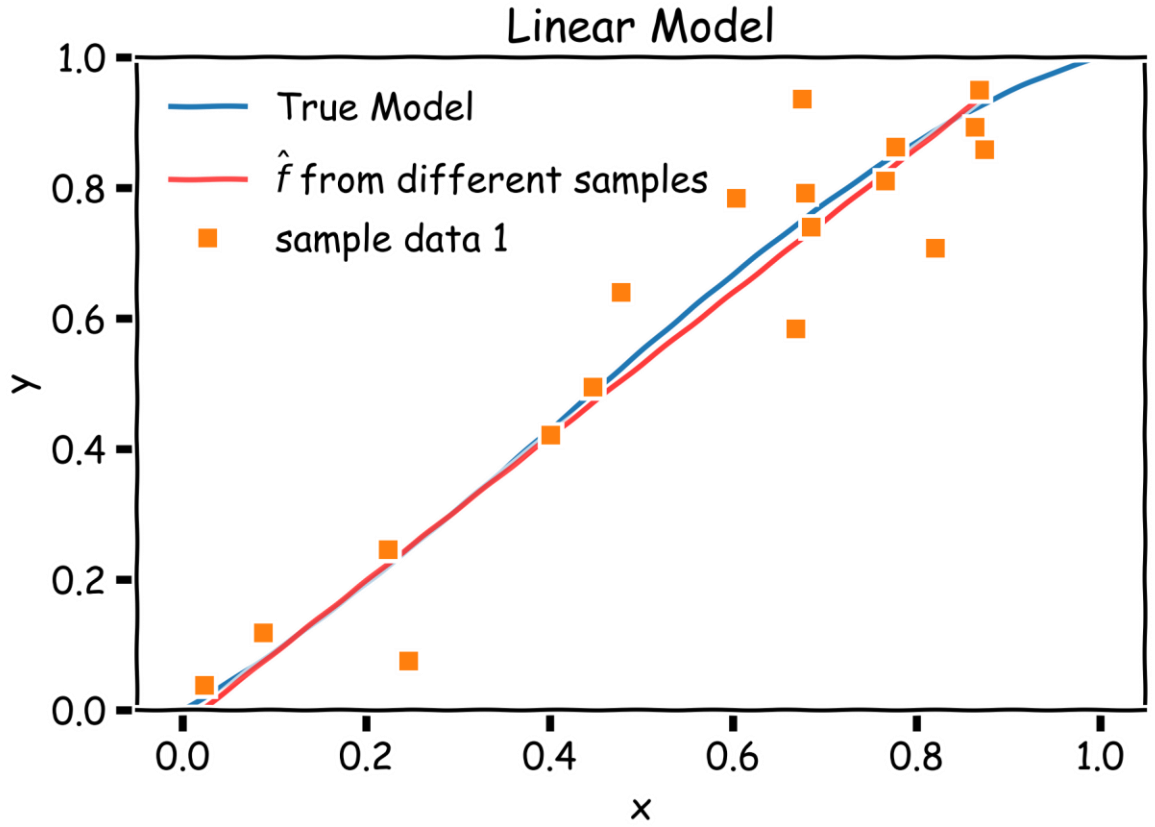
📌 *Scop*: modelul să nu memoreze zgomotul, ci să generalizeze.

Simplifici sau stabilizezi modelul:

- Reduci numărul de features (feature selection).
- Crești numărul de date pentru antrenament (augmentare sau colectare).
- Adaugi regularizare.

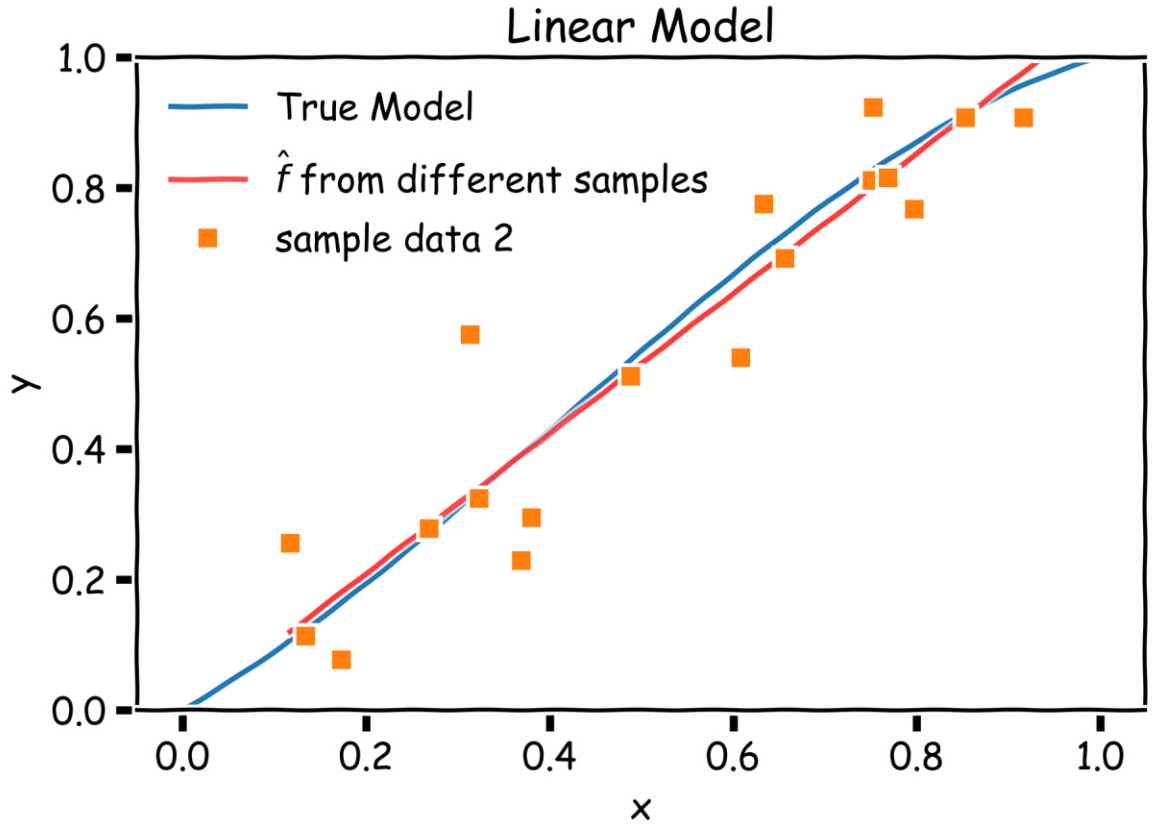
Bias vs Varianță

Simulare = subeșantionare aleatoare



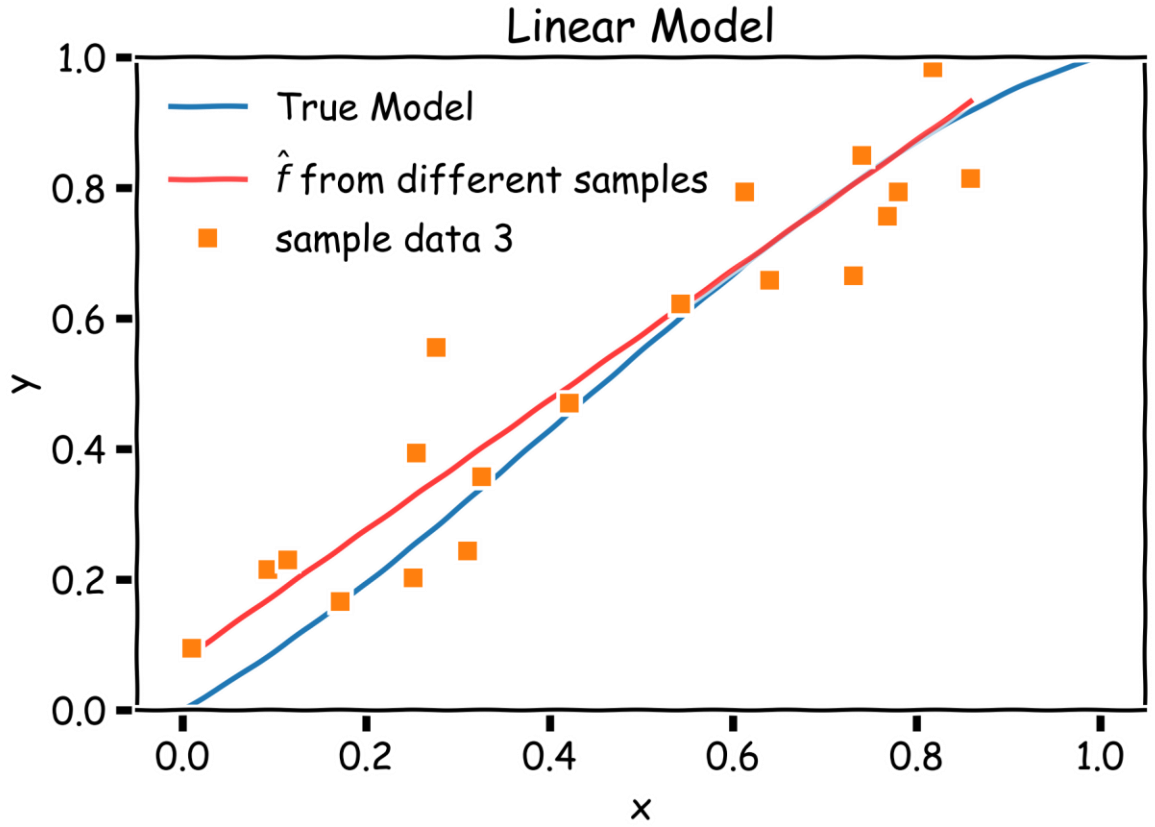
Bias vs Varianță

Simulare = subeșantionare aleatoare



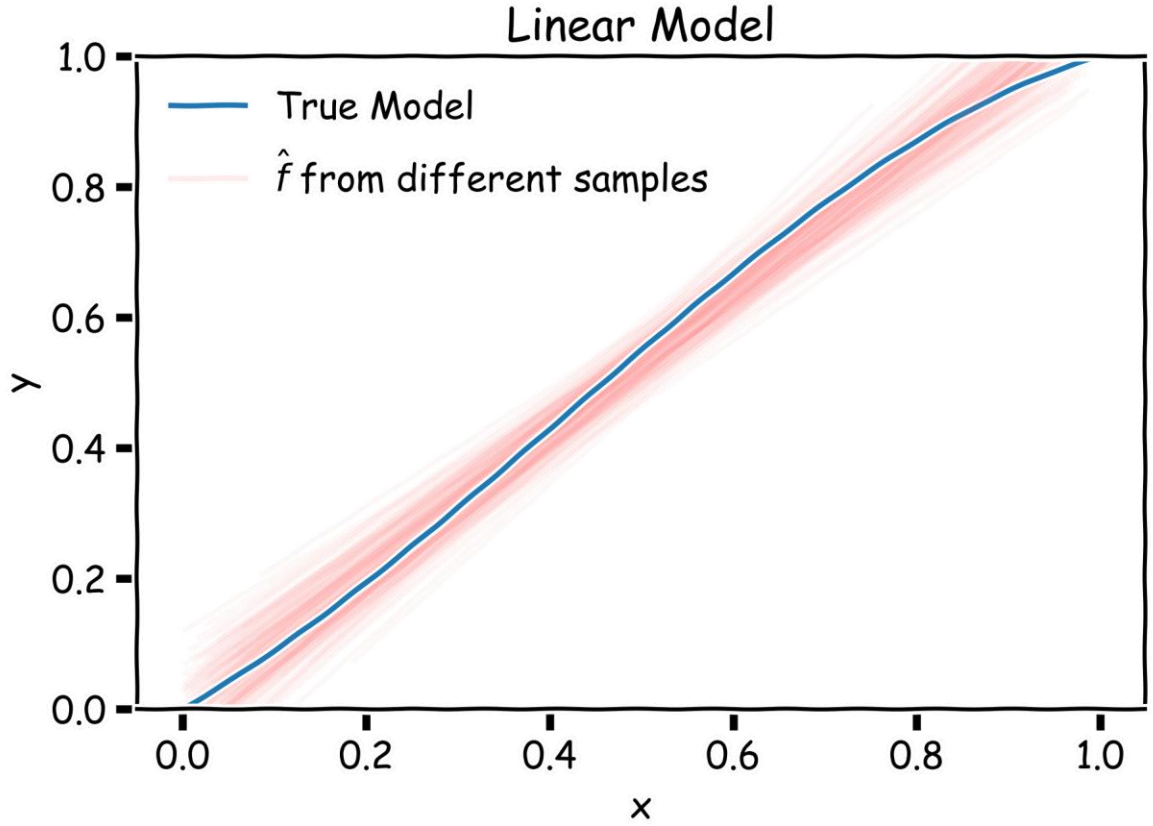
Bias vs Variance

Simulare = subeșantionare aleatoare



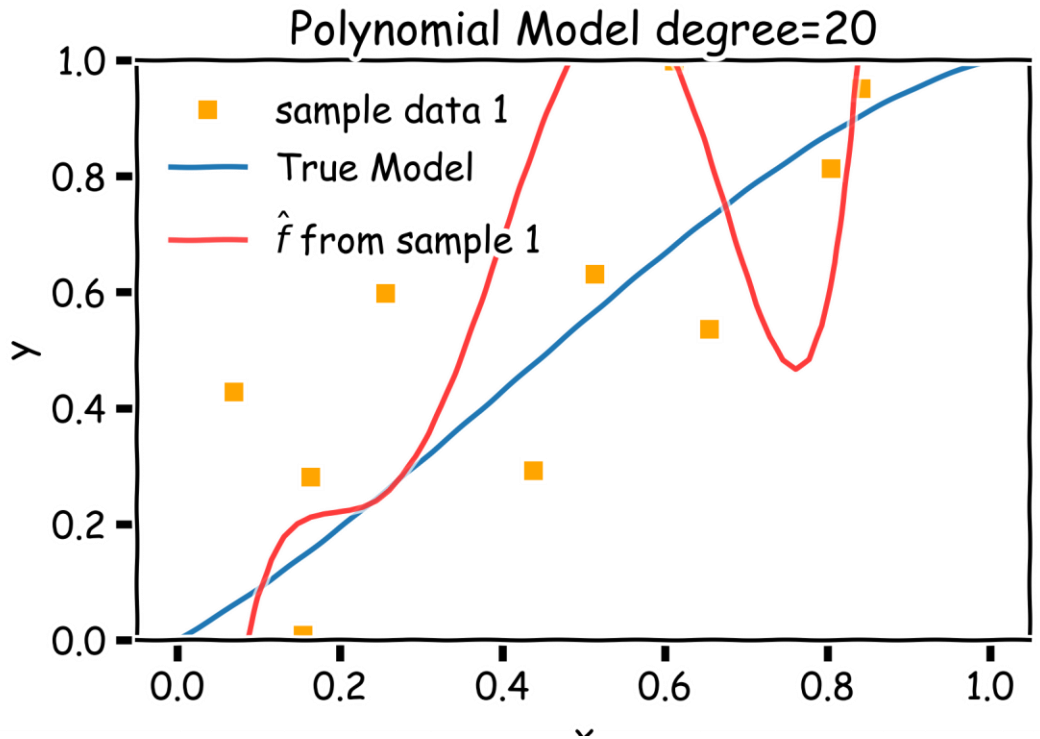
Modele liniare: 20 de înregistrări, 2000 simulări

Simulare = subeșantionare aleatoare

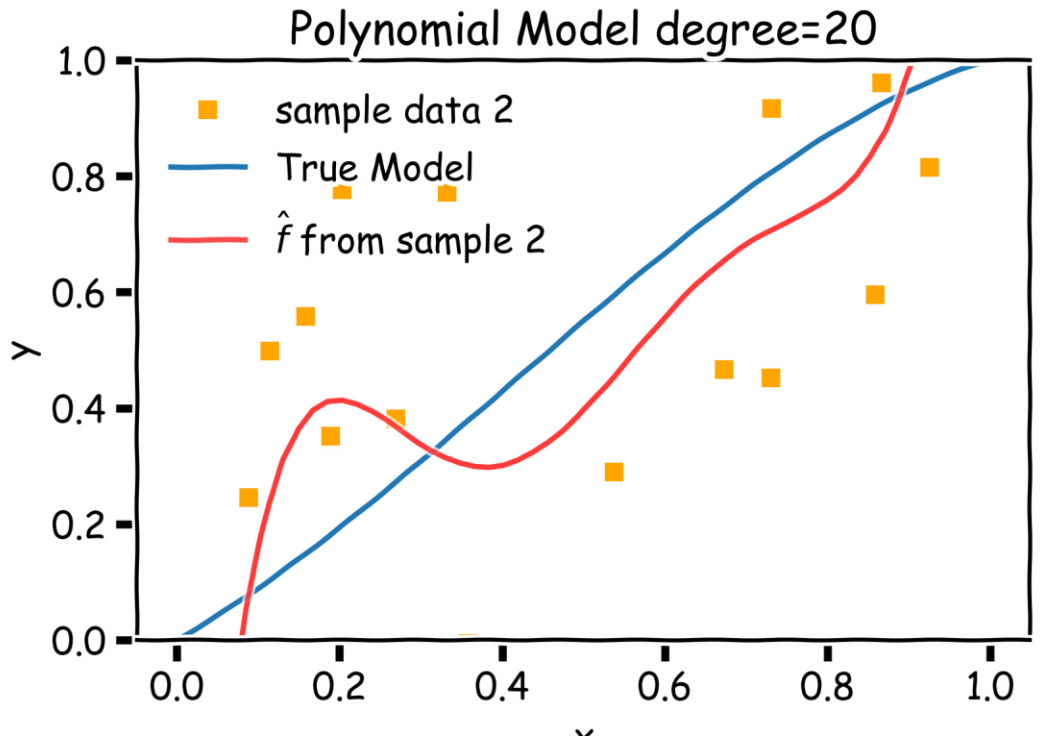


Bias vs Variance

Simulare = subeșantionare aleatoare

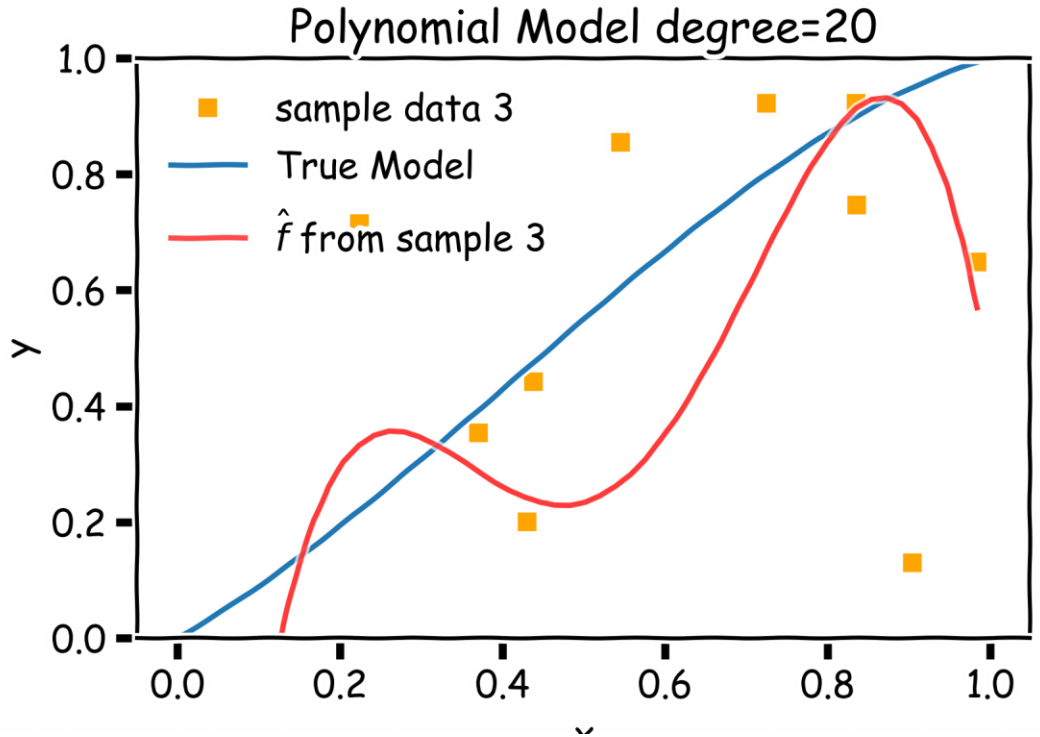


Bias vs Variance



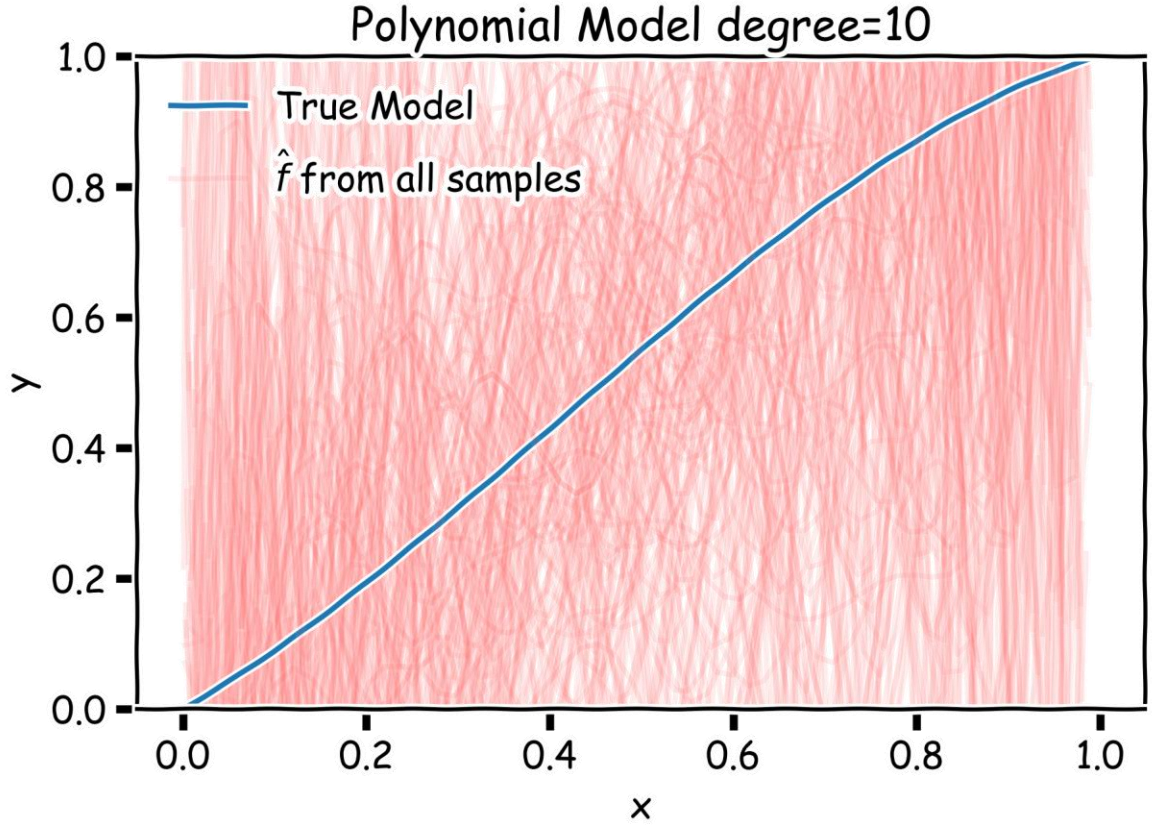
Bias vs Variance

Simulare = subeșantionare aleatoare



Polinom grad 10 : 20 de înregistrări, 2000 simulări

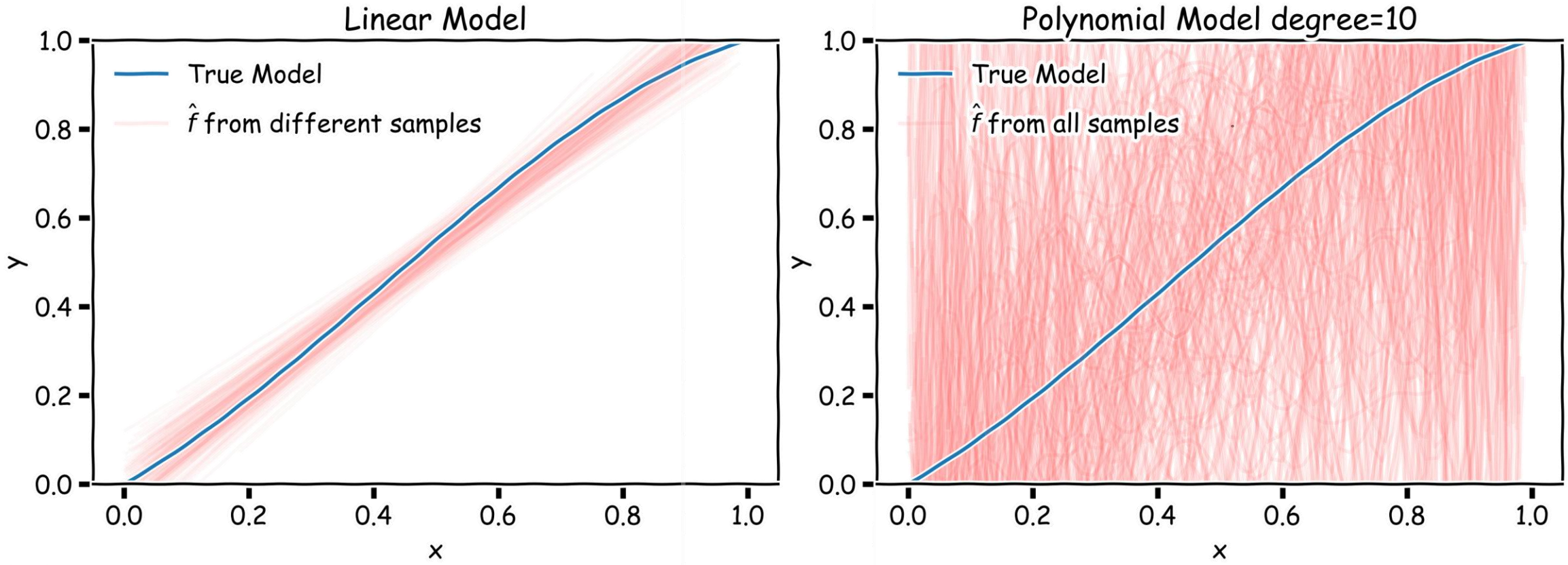
Simulare = subeșantionare aleatoare



Bias vs Variance

Stânga: 2000 drepte de regresie, fiecare dintr-un subeșantion diferit de 20 observații

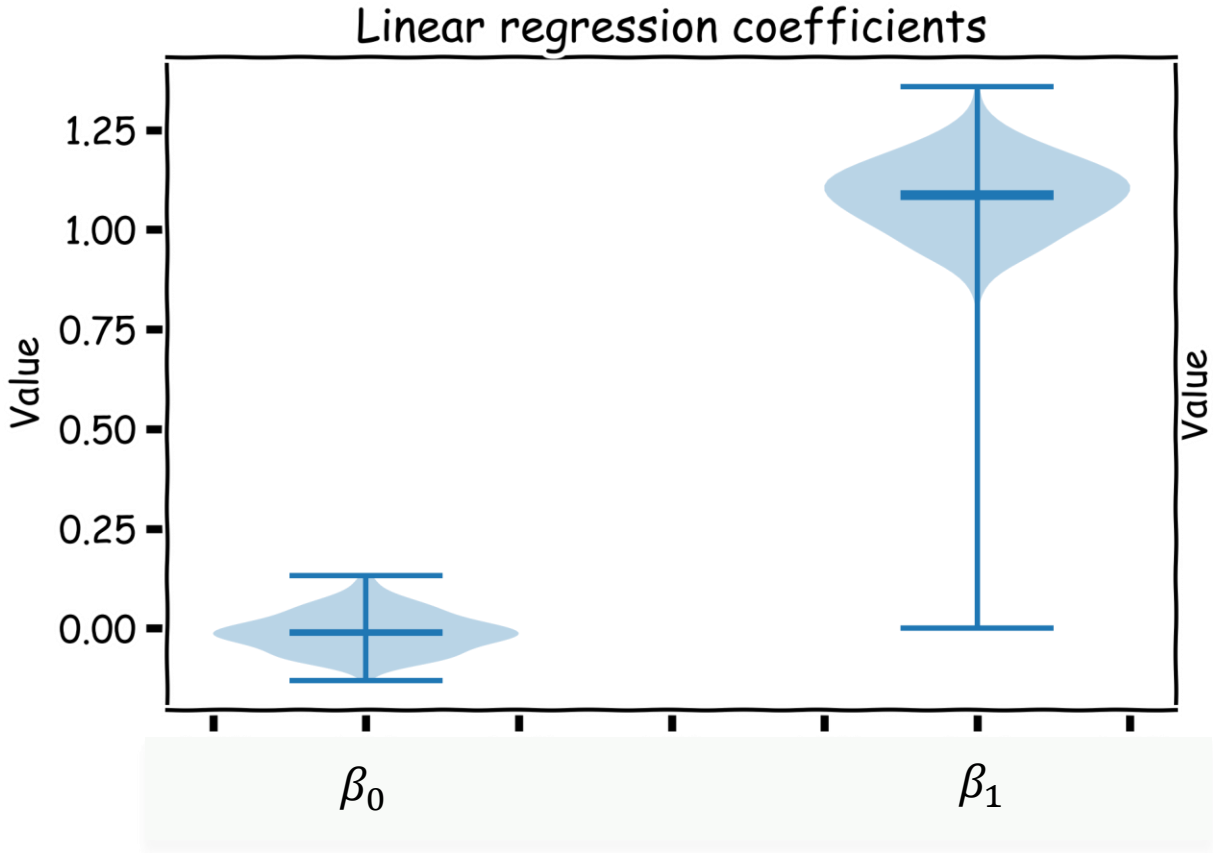
Dreapta: modele polinomiale de grad 10, pe aceleași subeșantioane

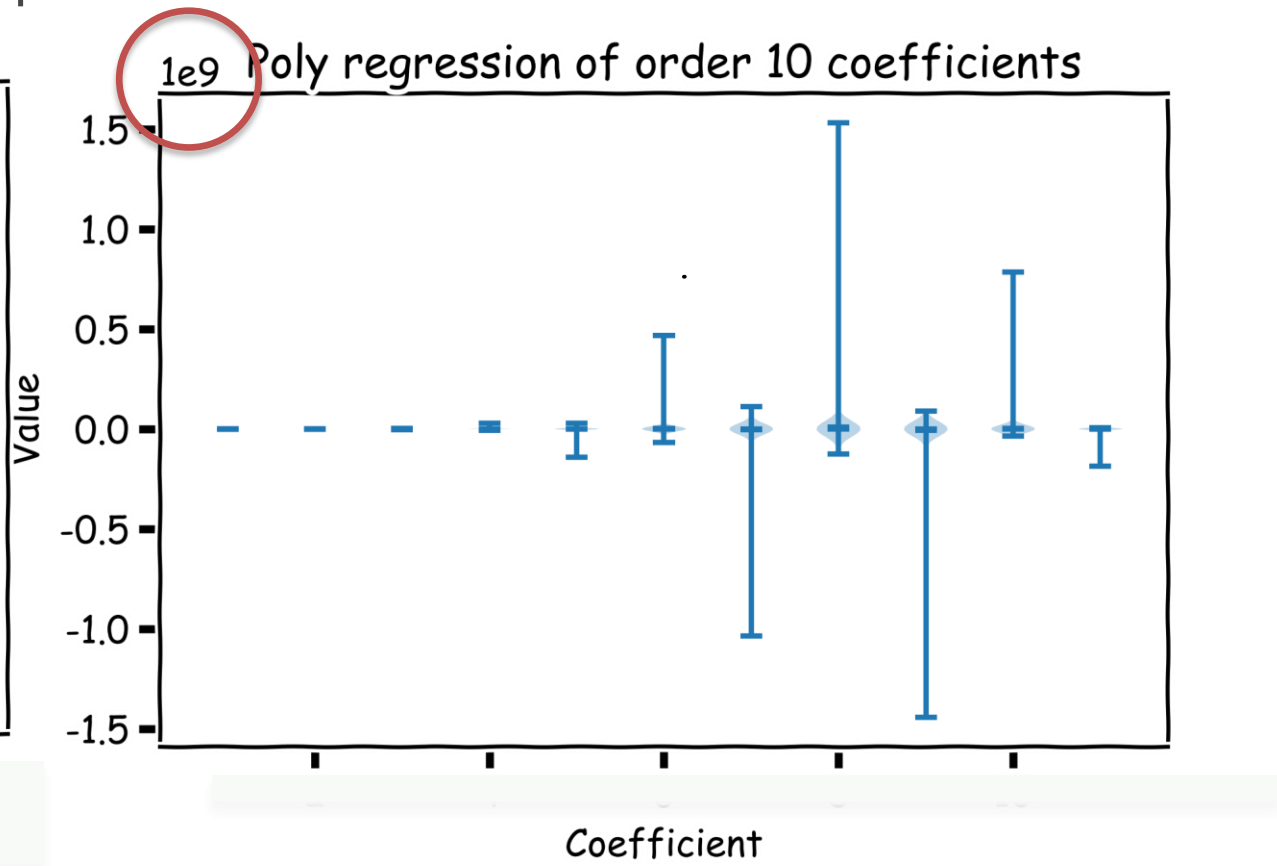


Bias vs Variance

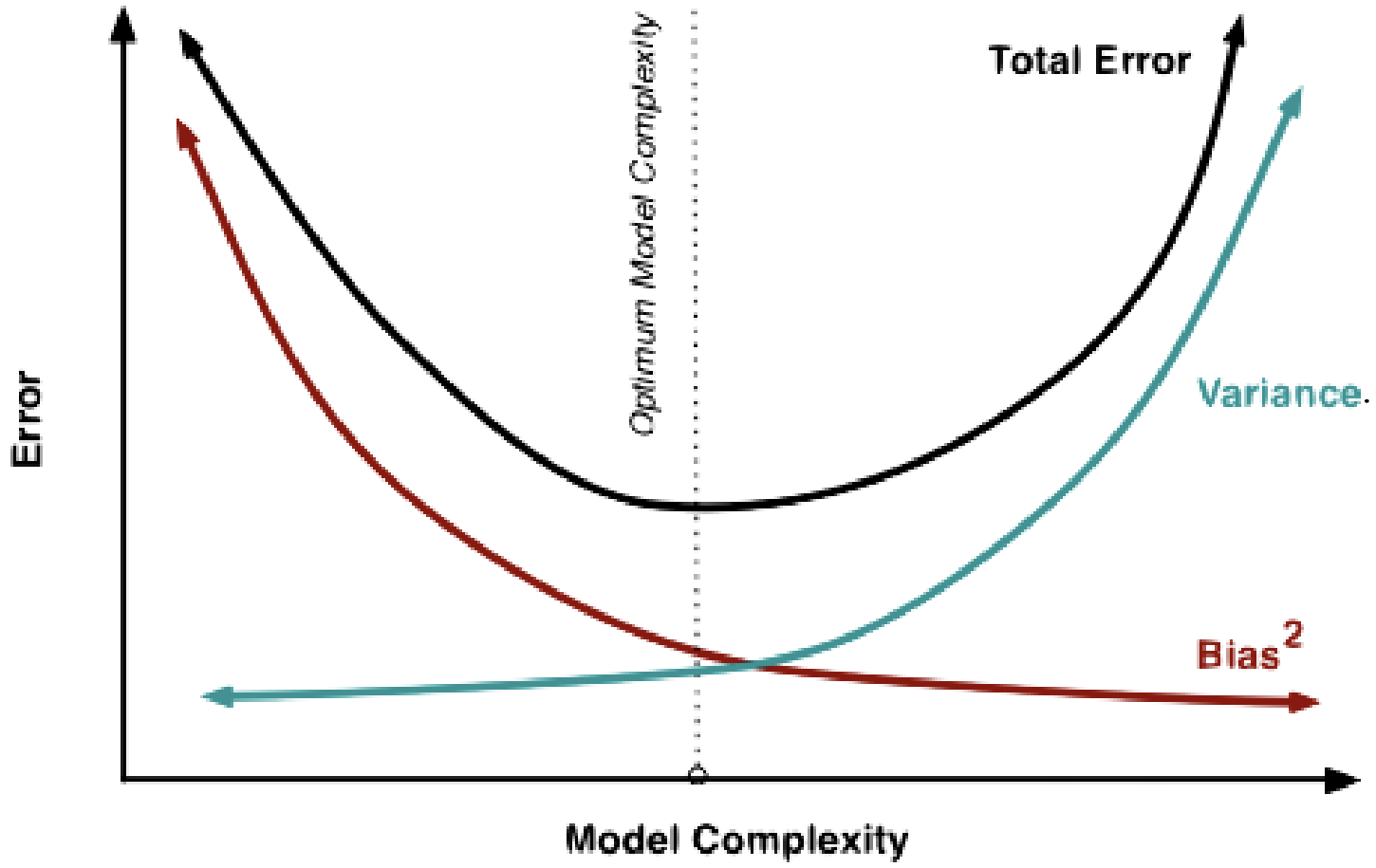
Stânga: box-plot Coeficienți regresie liniară

Dreapta: box-plot Coeficienți regresie polinomială de ordin 10





Bias vs Varianță - concluzie



REGULARIZARE: LASSO ȘI RIDGE

Regularizare: O imagine de ansamblu

Ideea de regularizare se învârte în jurul modificării funcției de pierdere L (loss function; de ex MSE); în special, adăugăm un termen de regularizare care penalizează unele proprietăți specificate ale parametrilor modeluluiȘ

$$L_{reg}(\beta) = L(\beta) + \lambda R(\beta),$$

unde λ este un scalar care dă greutatea (sau importanța) termenului de regularizare *R*.

Antrenarea modelului utilizând funcția de pierdere modificată L_{reg} ar duce la parametrii ai modelului cu proprietăți dorite (specificate de R).

Regresie LASSO

Deoarece dorim să descurajăm valorile extreme în parametrii modelului, trebuie să alegem un termen de regularizare care să penalizeze magnitudinea parametrilor. Pentru funcția noastră de pierdere vom folosi din nou MSE:

$$L_{LASSO}(\beta) = \frac{1}{n}\sum_{i=1}^{n}|y_i - \boldsymbol{\beta}^\top \boldsymbol{x}_i|^2 + \lambda\sum_{j=1}^{J}|\beta_j|.$$

Aici $\sum_{j=1}^{J}|\beta_j|$ este norma l_1 a vectorului $\boldsymbol{\beta}$

$$\sum_{j=1}^{J}|\beta_j| = \|\boldsymbol{\beta}\|_1$$

Regresie LASSO

Prin urmare, vom spune în mod uzual că L_{LASSO} este funcția de pierderi (loss-function) pentru regularizare l_1.

Găsirea parametrilor modelului, β_{LASSO} , care minimizează funcția de pierdere pentru regularizarea l_1 se numește regresie ***LASSO***.

In [ ]:
```
from sklearn.linear_model import Lasso
```

In [22]:
```
lasso_regression = Lasso(alpha=1.0, fit_intercept=True)
lasso_regression.fit(np.vstack((X_train, X_val)), np.hstack((y_train, y_val)))

print('Lasso regression model:\n {} + {}^T . x'.format(lasso_regression.intercept_, lasso_regression.coe
```

```
Lasso regression model:
 10.424895873901445 + [ 0.24482603  3.48164594  1.84836859 -0.06864603 -0.         -0.
 -0.02249766 -0.          0.          0.          0.          ]^T . x
```

In [23]:
```
print('Train R^2: {}, test R^2: {}'.format(lasso_regression.score(np.vstack((X_train, X_val)),
                                                                  np.hstack((y_train, y_val))),
                                           lasso_regression.score(X_test, y_test)))
```

```
Train R^2: 0.48154992527975765, test R^2: 0.6846451270316087
```

Regresie Ridge

Alternativ, putem alege un termen de regularizare care să penalizeze pătratele magitudinilor parametrilor. Astfel funcția noastră de pierdere regularizată este:

$$L_{Ridge}(\beta) = \frac{1}{n}\sum_{i=1}^{n} |y_i - \boldsymbol{\beta}^\top \boldsymbol{x}_i|^2 + \lambda \sum_{j=1}^{J} \beta_j^2 .$$

Aici $\sum_{j=1}^{J} |\beta_j|^2$ este norma l_2 a vectorului $\boldsymbol{\beta}$

$$\sum_{j=1}^{J} \beta_j^2 = \|\boldsymbol{\beta}\|_2^2$$

Ridge Regression

Prin urmare, vom spune în mod uzual că L_{ridge} este funcția de pierderi (loss-function) pentru regularizare l_2.

Găsirea parametrilor modelului β_{ridge} care minimizează funcția de pierdere pentru regularizarea l_2 se numește regresie ***ridge.***

In [ ]:
```
from sklearn.linear_model import Ridge
```

In [20]:
```
X_train = train[all_predictors].values
X_val = validation[all_predictors].values
X_test = test[all_predictors].values

ridge_regression = Ridge(alpha=1.0, fit_intercept=True)
ridge_regression.fit(np.vstack((X_train, X_val)), np.hstack((y_train, y_val)))

print('Ridge regression model:\n {} + {}^T . x'.format(ridge_regression.intercept_, ridge_regression.coe
```

```
Ridge regression model:
 -525.7662550875951 + [ 0.24007312  8.42566029  2.04098593 -0.04449172 -0.01227935  0.41902475
 -0.50397312 -4.47065168  4.99834262  0.          0.          0.29892679]^T . x
```

In [21]:
```
print('Train R^2: {}, test R^2: {}'.format(ridge_regression.score(np.vstack((X_train, X_val)),
                                                                  np.hstack((y_train, y_val))),
                                           ridge_regression.score(X_test, y_test)))
```

```
Train R^2: 0.5319764744847737, test R^2: 0.7881798111697319
```

Alegerea lui λ

Atât la regresia ridge cât și la cea LASSO, vedem că cu cât este mai mare alegerea parametrului de regularizare λ, cu atât penalizăm mai mult valorile mari în β

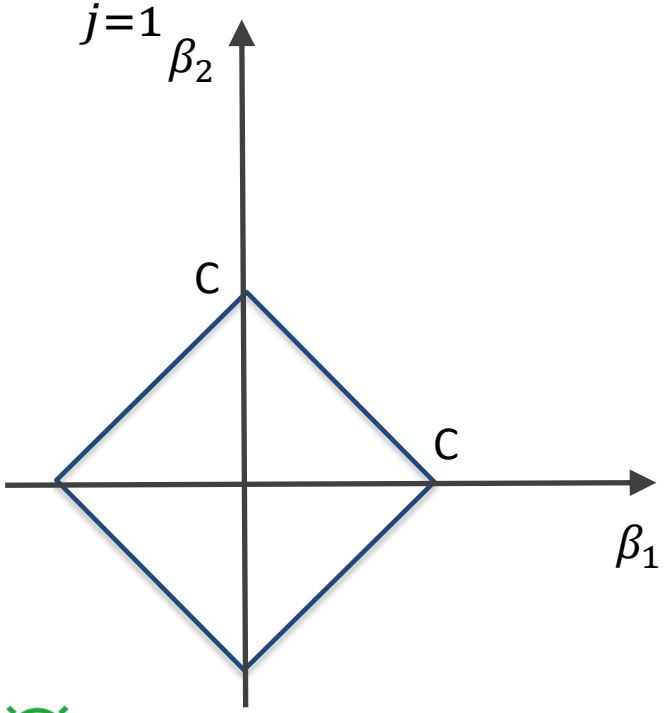
- Dacă λ e aproape de zero, regăsim MSE, adică regresiile ridge și LASSO regression se apropie de regresia obișnuită.
- Dacă λ este suficient de mare, termenul MSE în funcția de pierdere regularizată va fi nesemnificativ și termenul de regularizare β_{ridge} sau β_{LASSO} va fi forțat să se apropie de zero.

Pentru a evita alegeri ad-hoc pentru λ putem folosi validare încrucișată.

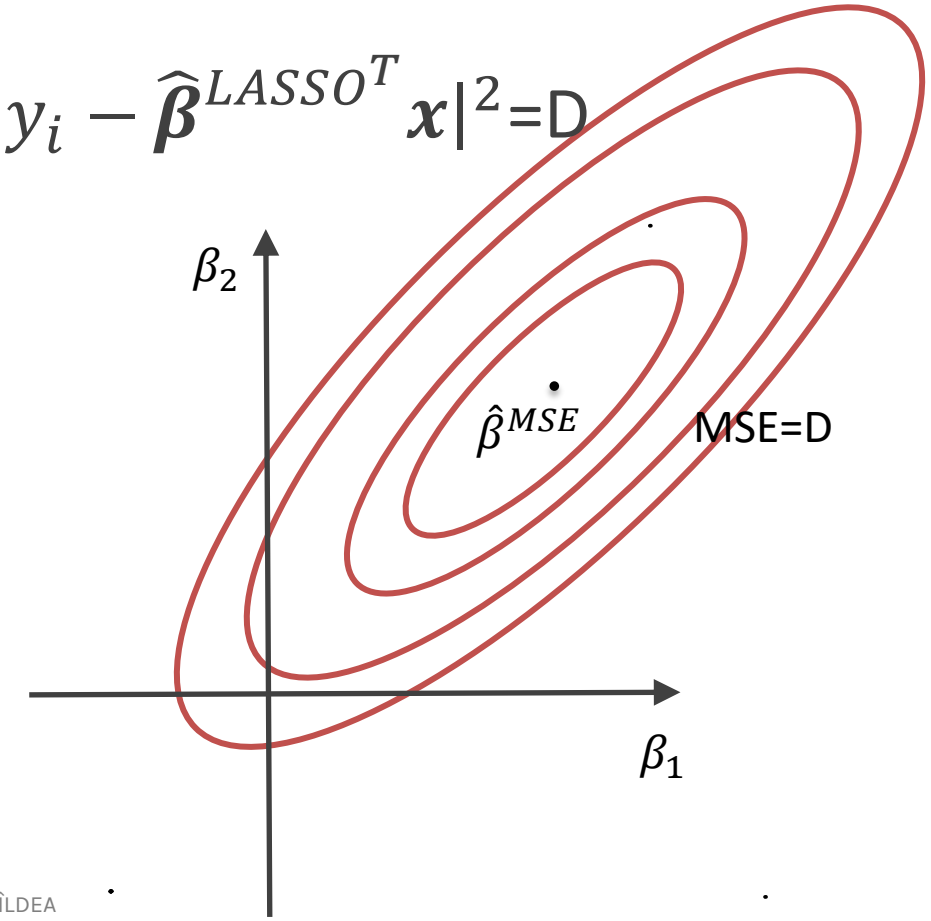
Geometria regularizării (LASSO)

$$L_{LASSO}(\boldsymbol{\beta}) = \frac{1}{n}\sum_{i=1}^{n} |y_i - \boldsymbol{\beta}^T \boldsymbol{x}|^2 + \lambda \sum_{j=1}^{J} |\beta_j| \qquad \widehat{\boldsymbol{\beta}}^{LASSO} = \operatorname{argmin} L_{LASSO}(\boldsymbol{\beta})$$

$$\lambda \sum_{j=1}^{J} |\hat{\beta}_j^{LASSO}| = C$$



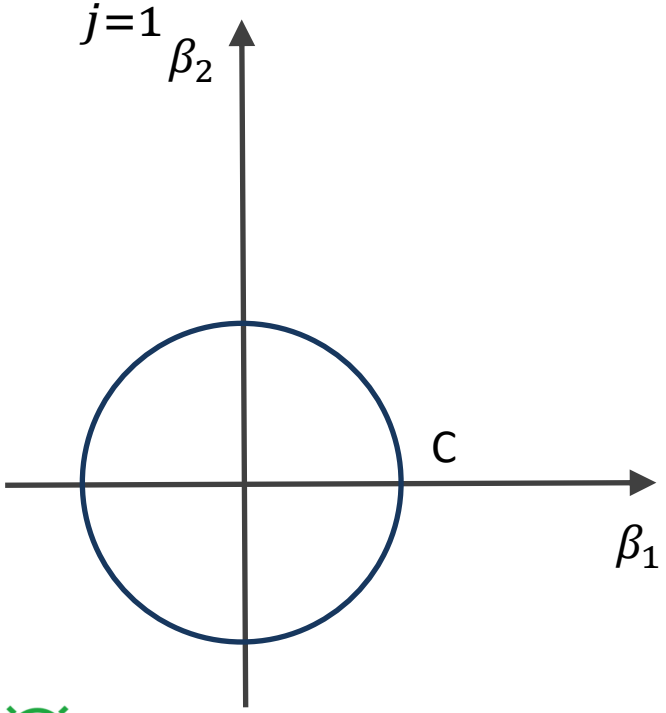
$$\frac{1}{n}\sum_{i=1}^{n} |y_i - \widehat{\boldsymbol{\beta}}^{LASSO^T} \boldsymbol{x}|^2 = D$$



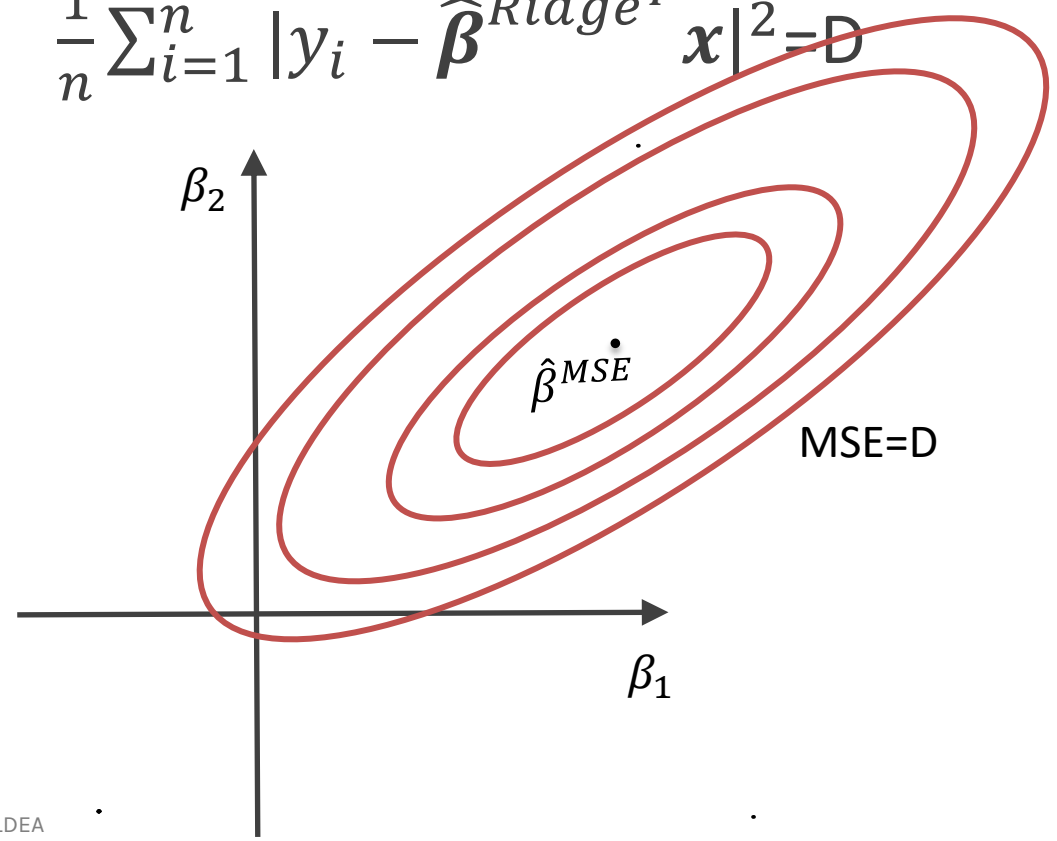
Geometria regularizării(Ridge)

$$L_{Ridge}(\boldsymbol{\beta}) = \frac{1}{n}\sum_{i=1}^{n}\left|y_i - \boldsymbol{\beta}^T\boldsymbol{x}\right|^2 + \lambda\sum_{j=1}^{J}\left(\beta_j\right)^2 \qquad \widehat{\boldsymbol{\beta}}^{Ridge} = \operatorname{argmin} L_{Ridge}(\boldsymbol{\beta})$$

$$\lambda\sum_{j=1}^{J}\left|\hat{\beta}_j^{Ridge}\right|^2 = C$$



$$\frac{1}{n}\sum_{i=1}^{n}\left|y_i - \widehat{\boldsymbol{\beta}}^{Ridge^T}\boldsymbol{x}\right|^2 = D$$



Ridge – Complexitate Computațională

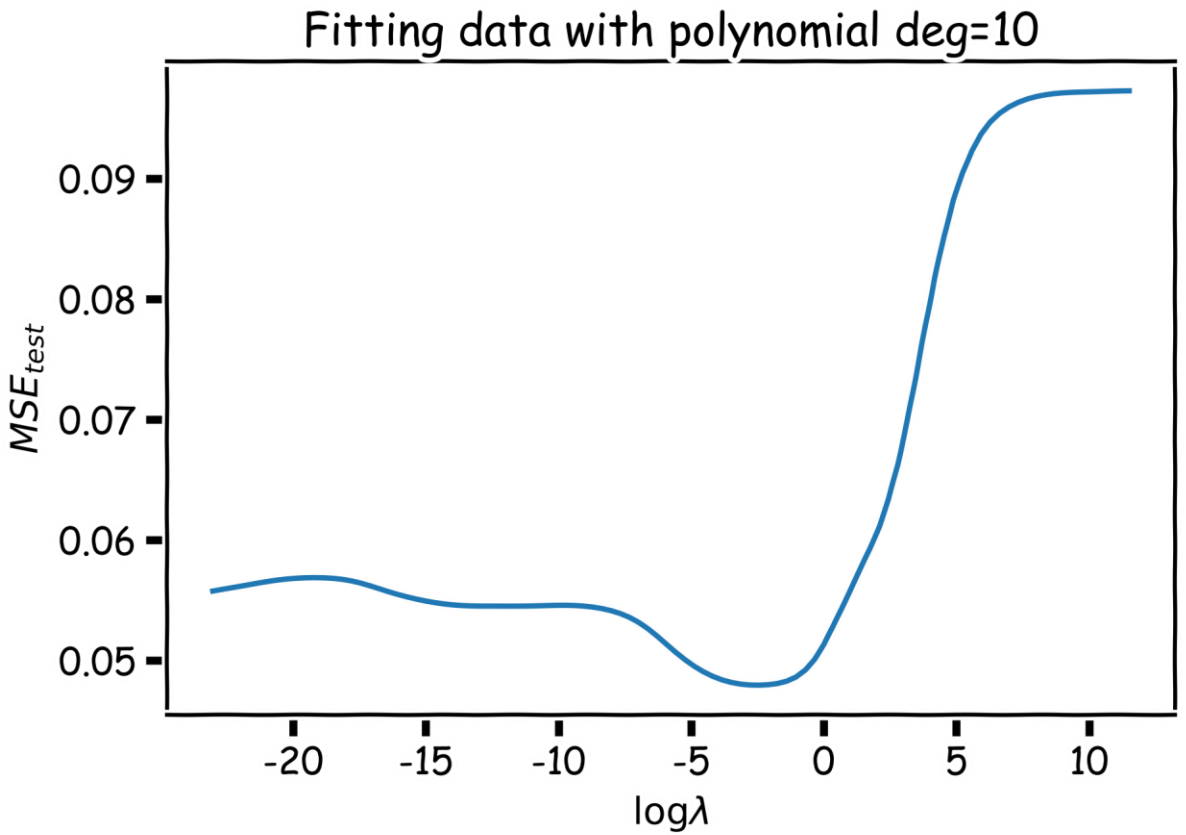
Soluția algebrică pentru regresia Ridge:

$$\beta = (X^T X + \lambda I)^{-1} X^T Y$$

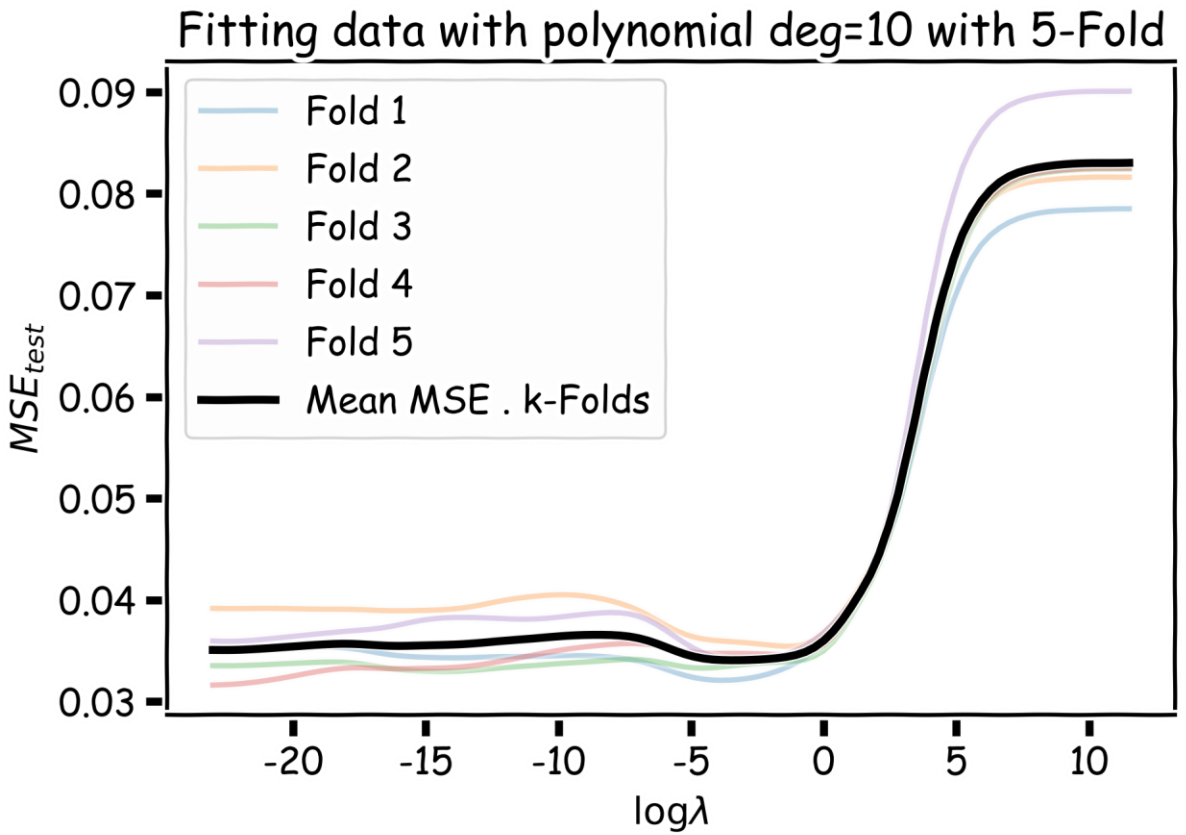
Soluția pentru regresia Ridge/Lasso presupune 3 pași

- Selectarea lui λ
- Găsirea minimului pentru funcția de pierdere ridge/Lasso regression (algebră liniară) și reținerea valorii R^2 **pe setul de date de test**.
- Găsirea acelui λ care ne dă cea mai mare valoare R^2

Ridge regularization with **validation** only: step by step



Regularizare Ridge cu **CV(Valid. Încr.)**: pas cu pas



Alegerea predictorilor prin regularizare

Deoarece regresia LASSO are tendința de a produce estimări zero pentru un număr de parametri ai modelului - spunem că soluțiile LASSO sunt rare - considerăm că **LASSO este o metodă pentru selecția predictorilor**.

Mulți preferă să utilizeze LASSO pentru selecția variabilă (precum și pentru suprimarea valorilor extreme ale parametrilor) în loc de selecție treptată, deoarece LASSO evită problemele statistice care apar în selecția treptată.

Întrebare:

Care sunt avantajele și dezavantajele celor două abordări?


Tableau
66.6K views
0:01 / 1:29

APPENDIX

Regularizare LASSO doar pe **validare**: pas cu pas

1. Împărțire date în $\{\{X,Y\}_{train},\{X,Y\}_{validation},\{X,Y\}_{test}\}$
2. Pentru $\lambda \text{ in } \{\lambda_{min},\dots \lambda_{max}\}$:
    1. determină β care minimizează L_{lasso}, $\hat{\beta}_{lasso}(\lambda),,$ folosind datele de antrenament.
    2. Reține $L_{MSE}(\lambda)$ folosind datele de validare.
3. selectarea lui λ care minimizează pierderea pe setul de validare

    $$\lambda_{lasso} = \text{argmin}_{\lambda}\, L_{MSE}(\lambda)$$

4. Reantrenarea modelului folosind reuniunea dintre setul de antrenament și validare,$\{\{X,Y\}_{train},\{X,Y\}_{validation}\}$, obținem $\widehat{\beta}_{lasso}(\lambda_{lasso})$
5. raportăm MSE sau R^2 pe $\{X,Y\}_{test}$ folosind $\widehat{\beta}_{lasso}(\lambda_{lasso})$

Regularizare Ridge cu **CV(Valid. Încr.)**: pas cu p

	λ_1	λ_2	...	λ_n
k_1	L_{11}	L_{12}	..	...
k_2	L_{21}	...	..	...
...	..	...	..	...
k_n	...	...	...	...
E[]	$\bar{L}_1$	$\bar{L}_2$	...	$\bar{L}_n$

1. Se elimină $\{X,Y\}_{test}$ din date
2. Spargerea datelor în K părți, $\{\{X,Y\}_{train}^{-k}, \{X,Y\}_{val}^{k}\}$
3. pentru k in $\{1,\dots,K\}$
    1. pentru λ in $\{\lambda_0, \dots, \lambda_n\}$:
        A. determină β care minimizează L_{ridge}, $\hat{\beta}_{ridge}(\lambda,k) = \left(\mathrm{X}^{\mathrm{T}}\mathrm{X} + \lambda I\right)^{-1} X^T Y$, folosind datele de antrenament, $\{X,Y\}_{train}^{-k}$.
        B. Reține $L_{MSE}(\lambda,k)$ datele de validare $\{X,Y\}_{val}^{k}$

        Aici vom avea o matrice 2-D, pe linii cu valori diferite ale lui k, coloanele pentru diferite valori ale lui λ.
4. Facem media $L_{MSE}(\lambda,k)$ pentru fiecare λ, $\bar{L}_{MSE}(\lambda)$.
5. Găsim λ care minimizează $\bar{L}_{MSE}(\lambda)$, -> λ_{ridge}.
6. Reantrenăm modelul folosind întregul set de antrenamenta, $\{\{X,Y\}_{train}, \{X,Y\}_{val}\}$, -> $\hat{\beta}_{ridge}(\lambda_{ridge})$
7. Raportează MSE sau R^2 pe $\{X,Y\}_{test}$ pentru $\hat{\beta}_{ridge}(\lambda_{ridge})$

Regularizare ridge doar pe **validare**: pas cu pas

1. Împărțire date în $\{\{X,Y\}_{train},\{X,Y\}_{validation},\{X,Y\}_{test}\}$
2. Pentru $\lambda \text{ in } \{\lambda_{min},\dots\lambda_{max}\}$:
    1. determină β care minimizează L_{ridge}, $\hat{\beta}_{Ridge}(\lambda) = \left(\mathrm{X}^{\mathrm{T}}\mathrm{X}+\lambda I\right)^{-1}X^{T}Y$, folosind datele de antrenament.
    2. Reține $L_{MSE}(\lambda)$ folosind datele de validare.
3. selectarea lui λ care minimizează pierderea pe setul de validare,
$$\lambda_{ridge} = \text{argmin}_{\lambda}\, L_{MSE}(\lambda)$$
4. Reantrenarea modelului folosind reuniunea dintre setul de antrenament și validare, $\{\{X,Y\}_{train},\{X,Y\}_{validation}\}$, obținem $\hat{\beta}_{ridge}\left(\lambda_{ridge}\right)$
5. raportăm MSE sau R^2 pe $\{X,Y\}_{test}$ folosind $\hat{\beta}_{ridge}\left(\lambda_{ridge}\right)$.